SC11: Water Quality Potability

Step 4 - Build a Fair Baseline

Beginner click-by-click guide with SC11 worked examples

Purpose

Compare the completed Step 3 ANN with one very simple method on the same validation and test rows. Keep the comparison honest, save evidence, then submit Step 4.

This is a worked reference for students doing SC01-SC10. Follow the process in your allotted project, but design your own project-specific baseline, code and evidence. Never submit SC11 files unchanged.

No command line is required for the student actions in this guide.

Start here - what Step 4 means

A **baseline** is a simple reference method. It must answer the same question as the Step 3 technique. If both methods see different data or use different measures, their scores cannot be compared fairly.

For SC11, the Step 3 ANN predicts the binary Potability label. The Step 4 baseline counts labels in the **training split only** and always predicts the majority label. The current SC11 training split is tied (894 label 0 and 894 label 1), so the documented tie rule predicts 0.

The reference program checks that each saved ANN prediction file matches the corresponding Step 2 validation or test CSV row-by-row before computing scores. It does not retrain the ANN or use the test labels to choose the baseline.

> **This is not Product V1**
>
> Step 4 establishes a fair benchmark and documents the result. Step 5 will connect the method to an input/output product interface.

Expected evidence

- A new **step-4-baseline** branch created from **step-3-ann-model**.
- A repeatable **src/compare_baseline.py** file and executed **notebooks/03_baseline_comparison.ipynb**.
- Metrics, a comparison CSV, row-aligned paired predictions, and one comparison figure.
- A completed **docs/STEP-4.md** with actual values, limitations and member contributions.
- A meaningful commit on GitHub and a successful Step 4 portal submission.

The Step 4 portal deadline is **20 October 2026, 11:59 PM IST**. The portal may accept later submissions, but lateness is recorded for later grading.

What can be reused - and what must be original

The Git sequence, fairness checklist and headings in STEP-4.md can be reused. Fill the headings with your own question, data, method, measurements and interpretation. The full SC11 script and notebook cells in the appendices are worked examples, not a universal solution.

Reusable structure	Project-specific work
Branch and commit sequence	Baseline rule and implementation
STEP-4.md headings and checklist	Dataset, paths, metrics and real results
Same-row comparison principle	Interpretation, limitations and team contributions

Appendix A gives a reusable STEP-4.md template. Appendix B is SC11's filled example. Appendix C gives complete SC11 Python source. Appendix D shows the SC11 notebook cells. Short excerpts appear beside selected worked examples.

Example file	SC11 location	Purpose
compare_baseline.py	src/compare_baseline.py	Repeatable comparison program
03_baseline_comparison.ipynb	notebooks/03_baseline_comparison.ipynb	Executed notebook with visible results
STEP-4.md	docs/STEP-4.md	Question, method, results and interpretation

What the program creates after a successful run

- data/reports/step4_baseline_metrics.csv - both methods' accuracy, precision, recall and F1 on validation and test.
- data/reports/step4_comparison.csv - ANN score minus baseline score for each measure.
- data/reports/step4_validation_paired_predictions.csv and step4_test_paired_predictions.csv - row-wise true label and both predictions.
- results/step4_baseline_vs_ann.png - side-by-side graph for validation and test.

> **Do not change Step 3**
>
> Keep train_ann.py, the saved Step 3 metrics, predictions, notebook and model. The new comparison must use the exact saved ANN predictions and the exact same Step 2 split rows.

Action 1: Open the existing project

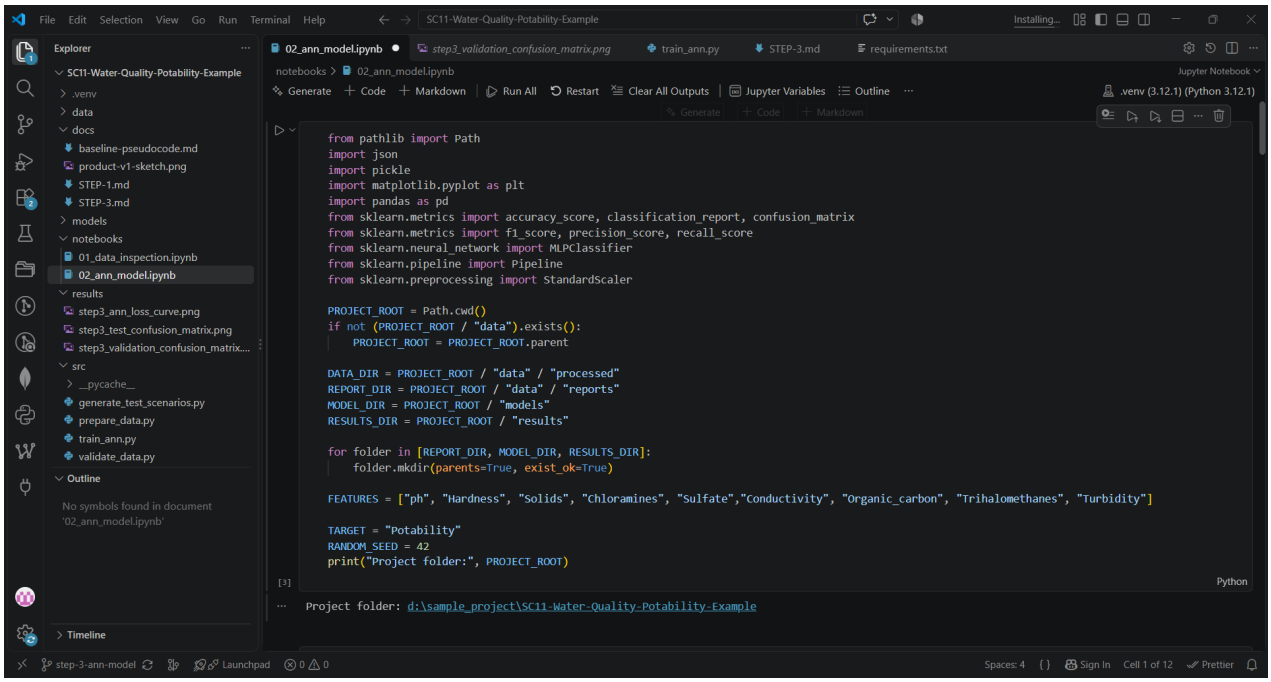
Why: Continue from completed Step 3 without making a second local copy.

1. Open VS Code. Choose File > Open Folder if SC11 is not already open.
2. Choose your existing SC11-Water-Quality-Potability-Example folder on this computer.
3. In Explorer, expand data, notebooks, src, docs and results. Do not delete Step 3 files.
4. Look at the branch name at bottom-left. It should be step-3-ann-model before creating Step 4.

What to check: The existing 02_ann_model.ipynb, train_ann.py and Step 3 report files are still visible.

If something goes wrong: If you are on a new computer with no project folder, use VS Code's Clone Repository button with the GitHub HTTPS URL, then switch to step-3-ann-model.

SC11 Step 3 project open in VS Code



Action 2: Create the Step 4 branch from Step 3

Why: A branch preserves Step 3 and makes Step 4 reviewable as one unit.

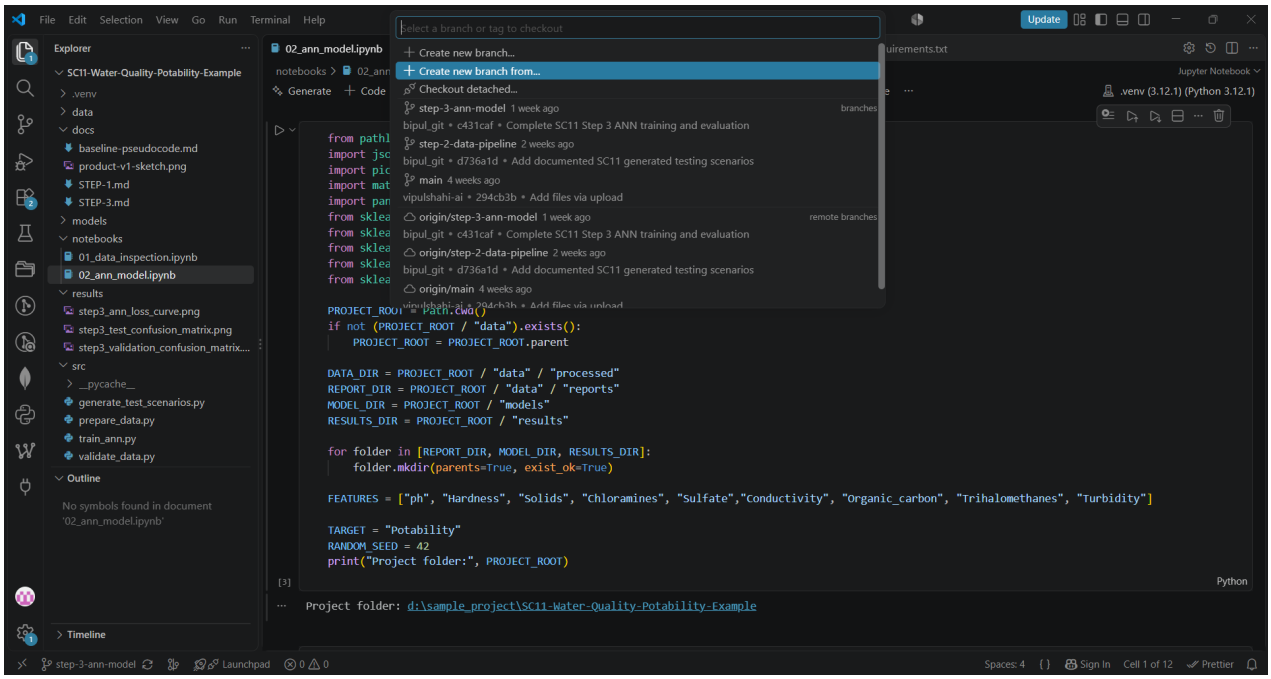
1. Click the branch name at the bottom-left of VS Code.
2. Choose **Create new branch from...**, not a branch from an unknown starting point.
3. Select **step-3-ann-model** as the source branch.
4. Type **step-4-baseline** and press Enter.
5. Confirm the bottom-left status bar now says step-4-baseline.

What to check: The new branch has all Step 2 and Step 3 files; no work was copied manually.

If something goes wrong: If Step 3 is not listed, use Source Control's Sync Changes first and check GitHub for the branch.

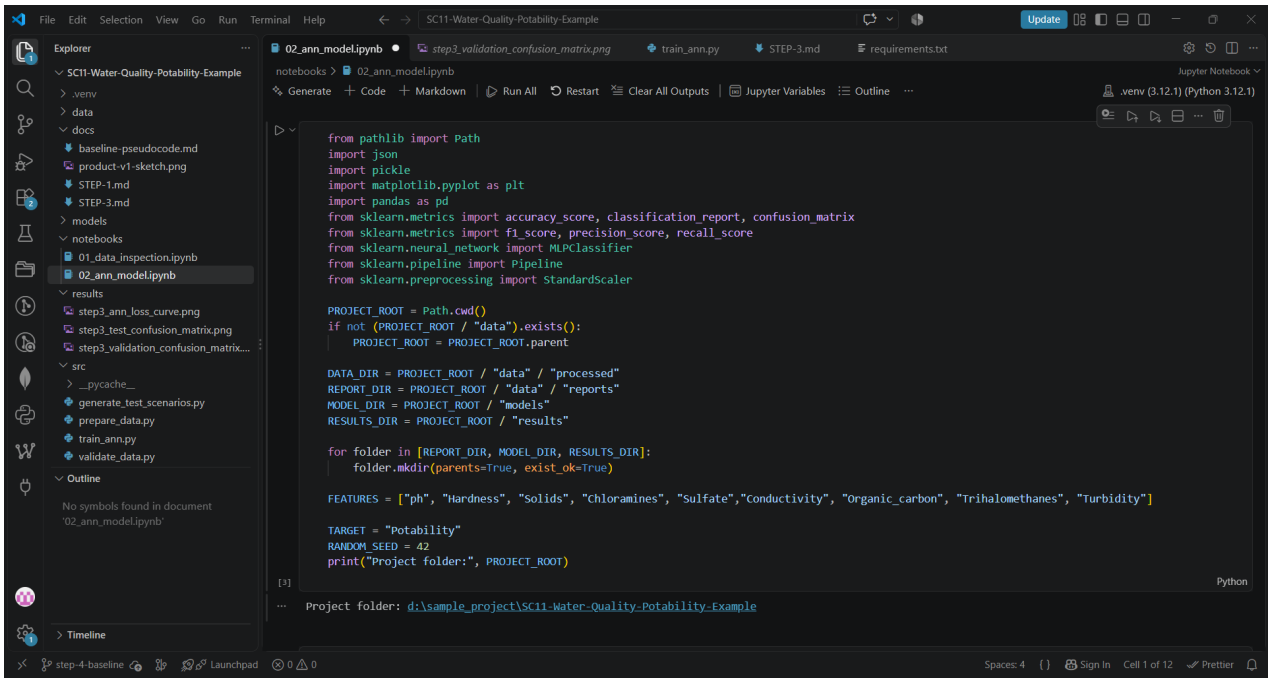
Action 2 - SC11 worked example

Choose Create new branch from...



Action 2 - SC11 worked example (continued)

New step-4-baseline branch visible



Action 3: Check the Step 3 comparison inputs

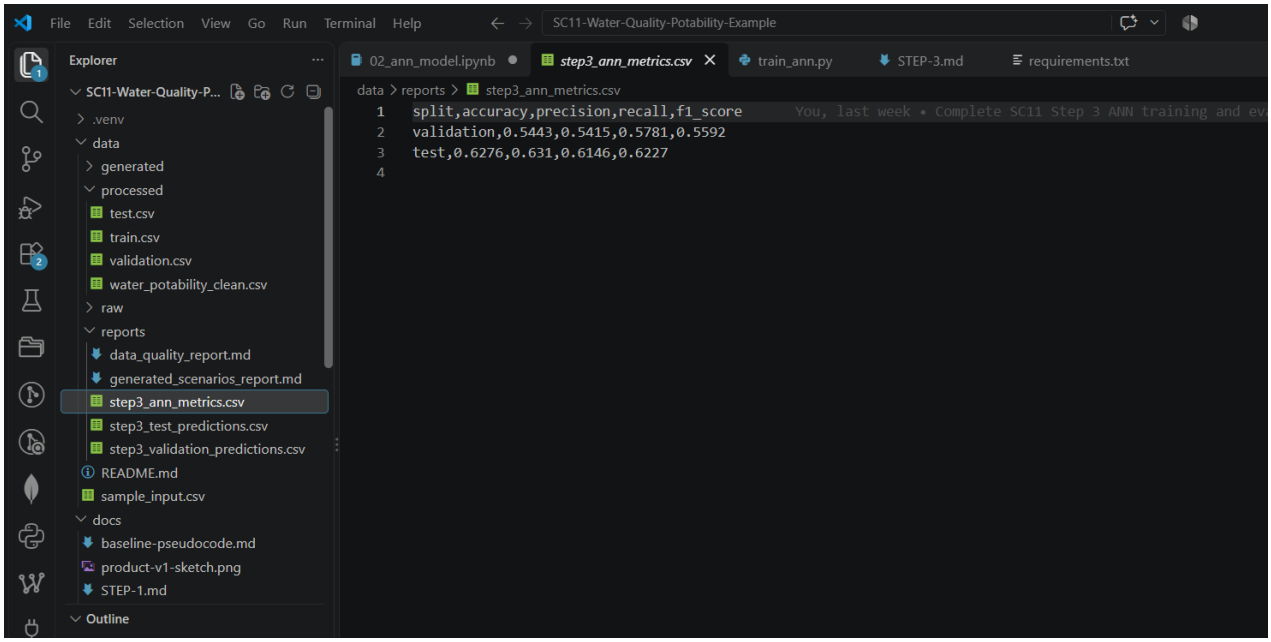
Why: The baseline must be compared to the ANN on identical known rows.

1. Open data/processed/train.csv, validation.csv and test.csv in VS Code Explorer.
2. Open data/reports/step3_validation_predictions.csv and step3_test_predictions.csv.
3. Open data/reports/step3_ann_metrics.csv; note validation and test scores.
4. Do not use data/generated scenarios as if they were real labelled records.

What to check: SC11 has 1,788 training rows and 384 rows in each held-out split; your own project may differ.

If something goes wrong: If any file is missing, finish or sync Step 3 before continuing.

Existing Step 3 metrics and split files



Action 4: Write the baseline question before coding

Why: You need one clear question and one rule to avoid changing the method after seeing test results.

1. In VS Code Explorer, right-click **docs**, choose **New File**, and name it exactly **STEP-4.md**.
2. Use the reusable headings in Appendix A as a template. Replace every bracketed prompt with your own project's question, dataset, method and measures.
3. SC11 asks whether an ANN beats a training-majority baseline on identical held-out rows. Your team should define an equivalent project-specific question and simple rule before seeing test results.
4. Save with **Ctrl+S**. Leave result values blank until you have actual output.

Shared STEP-4.md headings - fill with your own project evidence

```
# Step 4 - Fair comparison report
## Team and project
## Question
## Data and scenarios
## Baseline rule
## Fair-comparison checks
## Results
## Interpretation
## Limitations and responsible use
## Reproducibility and evidence
## Team contributions
```

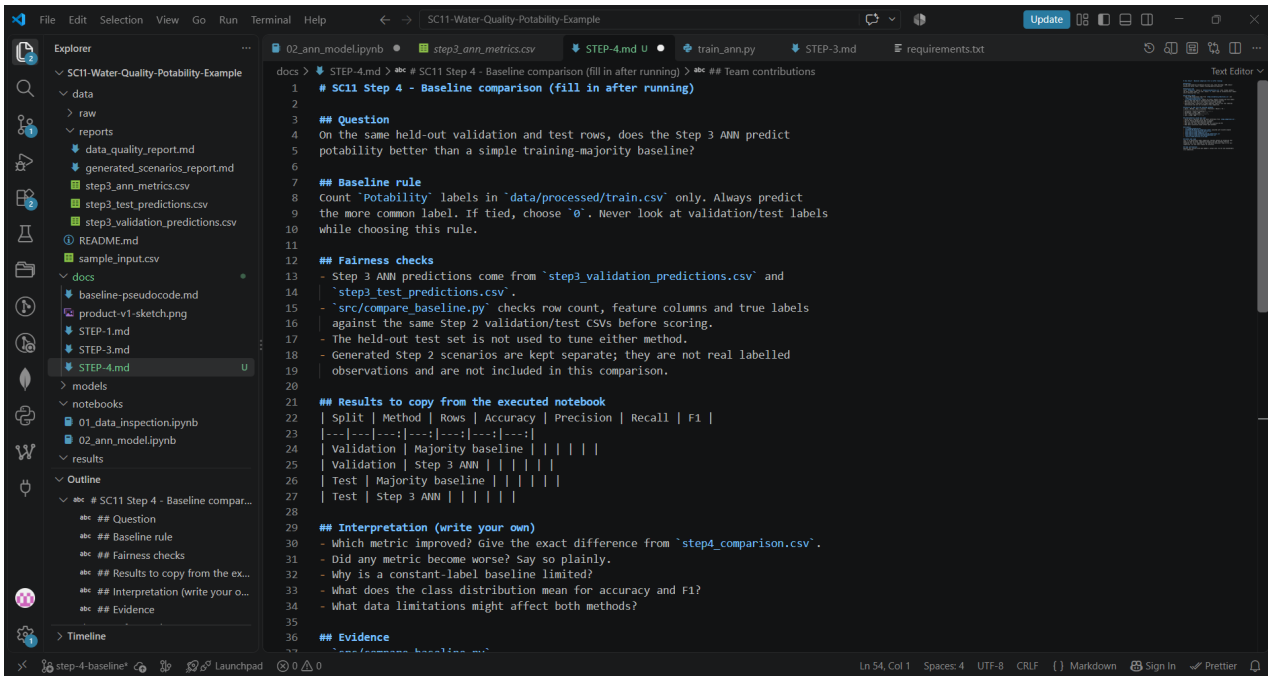
What to check: The file contains your question, baseline rule, fairness checks, blank results table, evidence list, limitations and contributions prompt.

If something goes wrong: Do not fill results with guessed numbers or choose the label by looking at validation/test targets.

Action 4 - SC11 worked example

Read this as an SC11 example: its water-quality question, Potability label and file names are not instructions for other projects. Use the generic template in Appendix A for your own report.

SC11 report example, not a universal template



Action 5: Create the repeatable Python file

Why: The .py file lets another person reproduce the comparison without your notebook clicks.

1. In Explorer, right-click src and choose New File.
2. Name it exactly compare_baseline.py.
3. Study the short excerpt below and the complete SC11 program in Appendix C. Write your own baseline using your Step 2 data and Step 3 outputs.
4. Keep your own folder structure and document the paths you use. The SC11 paths may not match your project.
5. Save the script with a descriptive name and explain its rule in STEP-4.md.

SC11 code excerpt - adapt the idea to your own project

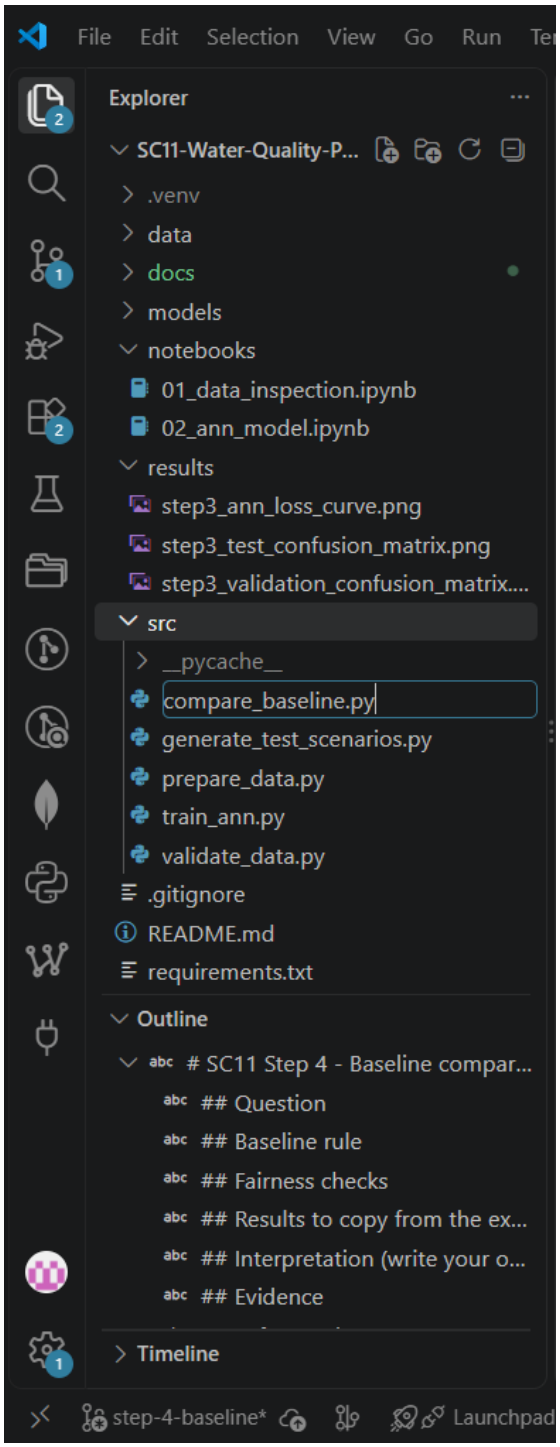
```
# SC11 example: select a rule using training labels only
counts = train['Potability'].value_counts()
baseline_label = int(counts.idxmax())
# Replace Potability and the rule for your project.
```

What to check: Your file includes imports, a project-specific baseline function, comparison and output-saving code.

If something goes wrong: If Windows hides extensions, check that the file is not compare_baseline.py.txt.

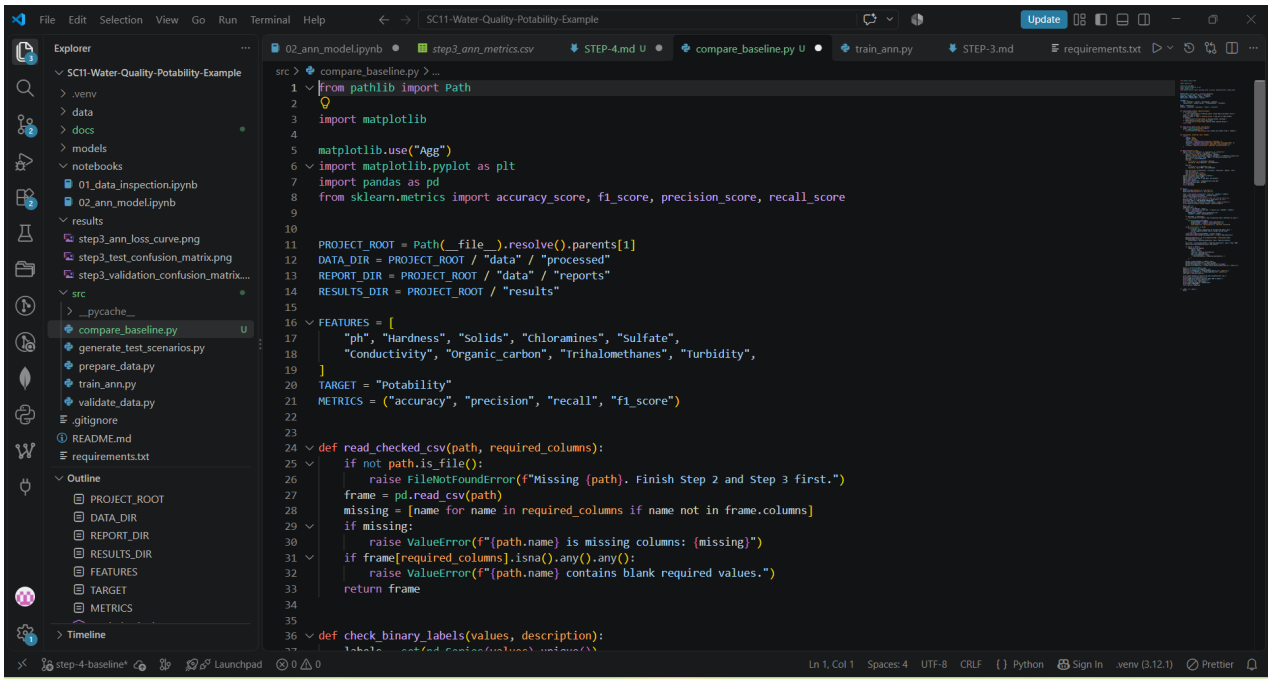
Action 5 - SC11 worked example

Create compare_baseline.py inside src



Action 5 - SC11 worked example (continued)

SC11 comparison script with paths and features



Action 6: Understand the fairness check in code

Why: A score comparison is invalid if one method's prediction rows do not match the original split.

1. Open **src/compare_baseline.py** and find **read_checked_csv**. It checks that required CSV files, columns and values exist.
2. In **main()**, find **train.csv**. The baseline label is selected only from the training target, before validation or test rows are loaded.
3. Next find **pd.testing.assert_frame_equal**. It compares the Step 3 saved prediction rows with the current Step 2 split, including features and true labels.
4. The SC11 script stops if rows, features or labels differ; its exact source is in Appendix C. Implement an equivalent check for your project's comparison.
5. Do not bypass an error by deleting the check. Fix the underlying files or branch instead.

SC11 code excerpt - adapt the idea to your own project

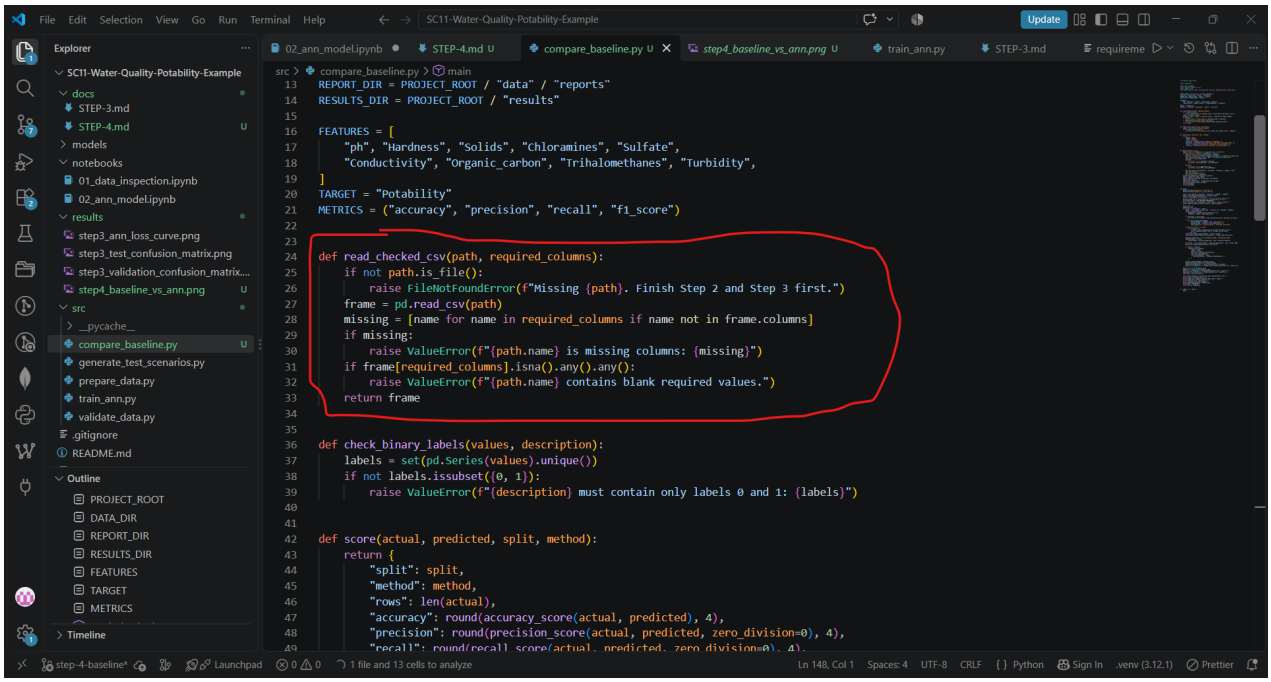
```
# SC11 example: verify the same rows were scored
pd.testing.assert_frame_equal(actual_rows, saved_rows)
```

What to check: The code does not load an untrusted model file or retrain the ANN.

If something goes wrong: If a mismatch appears later, verify the Step 3 prediction CSVs were produced from these exact split files.

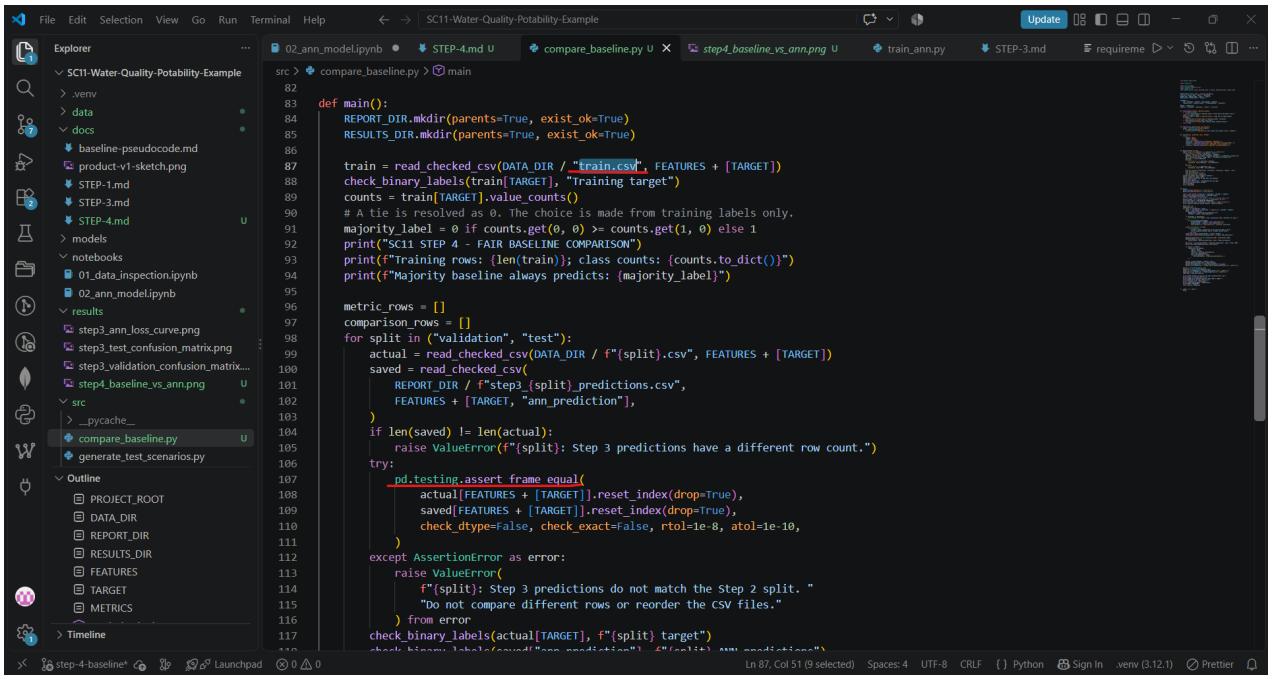
Action 6 - SC11 worked example

Required-file and column validation



Action 6 - SC11 worked example (continued)

Matching saved predictions to the held-out split



Action 7: Run the Python file in VS Code

Why: This produces the metrics, paired predictions and comparison chart.

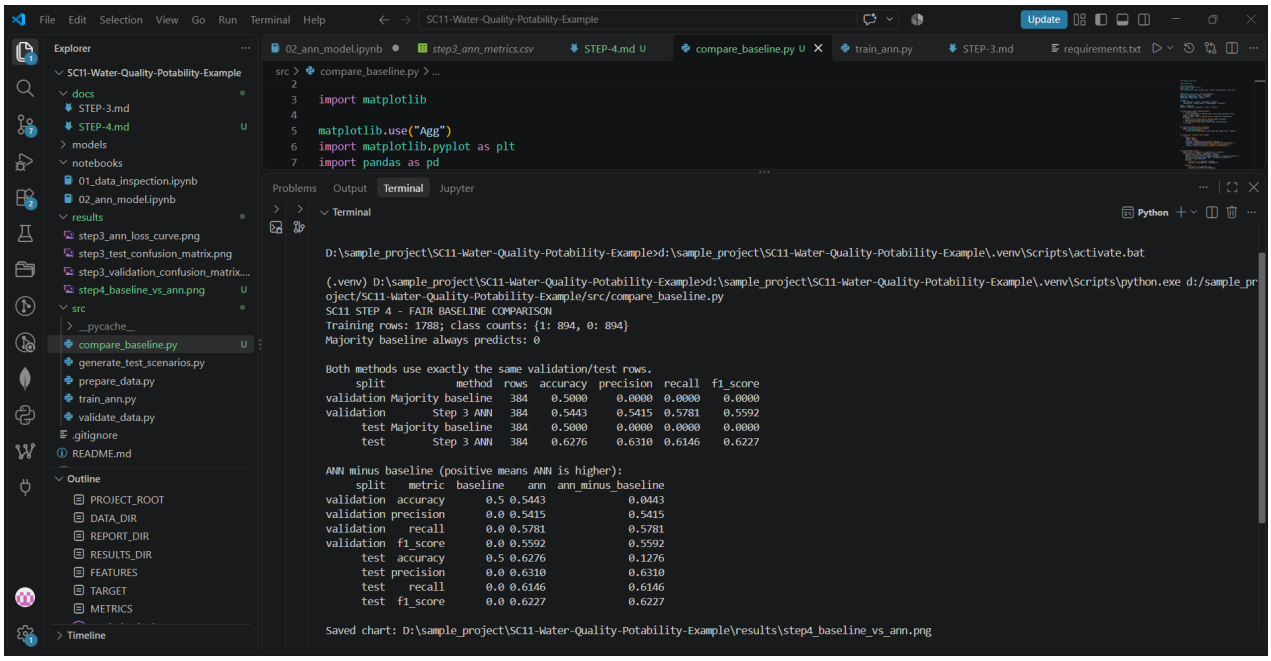
1. Open src/compare_baseline.py in the editor.
2. At the top-right, click **Run Python File**. Use the same Python interpreter as Step 3.
3. Wait for the terminal output inside VS Code. You do not need to type a command.
4. Read the training class counts, baseline label and the final **STEP 4 COMPLETED** message. If an error appears, stop and solve it before proceeding.
5. Refresh Explorer if needed. Confirm **data/reports** now has four new Step 4 CSV files and **results** has the new PNG chart.

What to check: A successful run reports 894 label-0 and 894 label-1 training rows for the current SC11 files; tie rule predicts 0.

If something goes wrong: If a package is missing, select the Python environment used in Step 3; do not change the CSV files to hide the error.

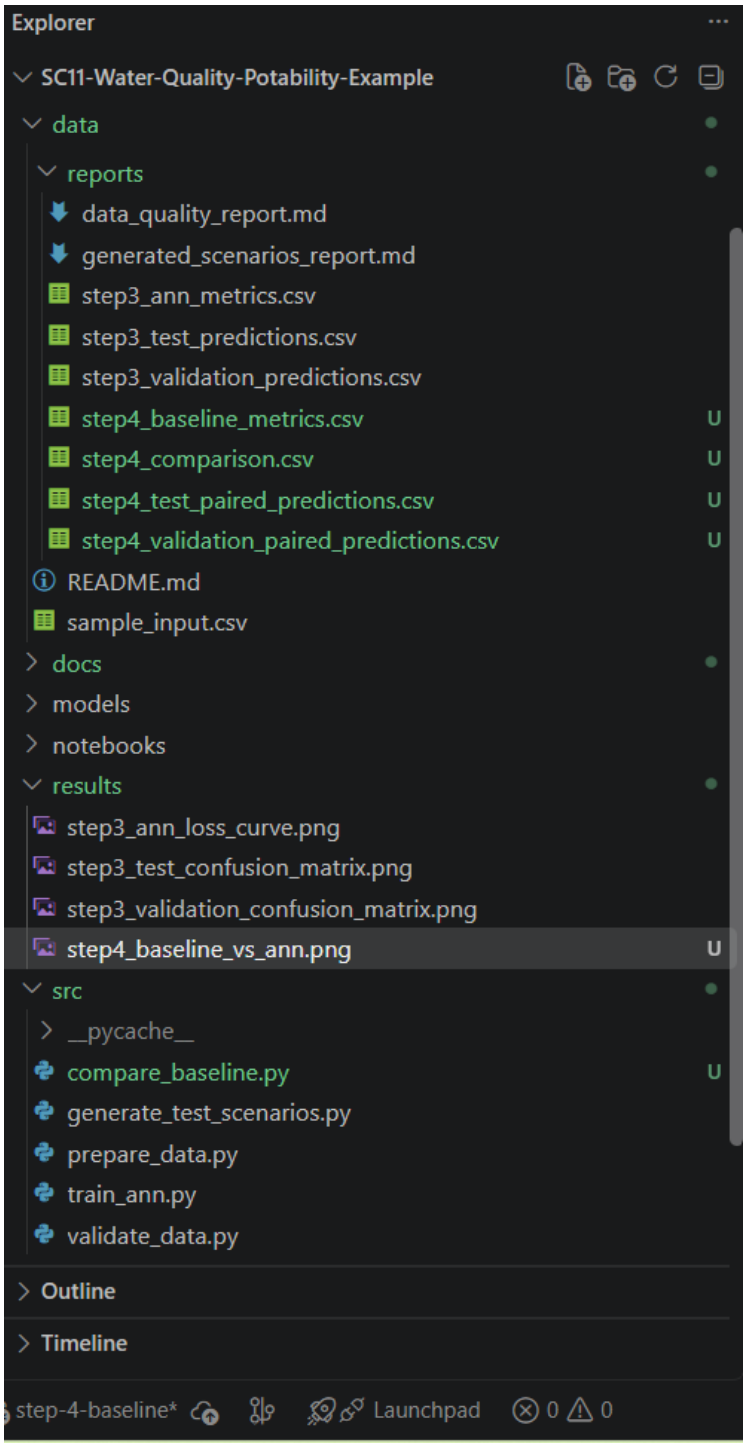
Action 7 - SC11 worked example

Successful run with validation and test metrics



Action 7 - SC11 worked example (continued)

New CSV reports and comparison figure in Explorer



Action 8: Read the scores honestly

Why: A single accuracy number can hide the fact that the baseline never predicts potable water.

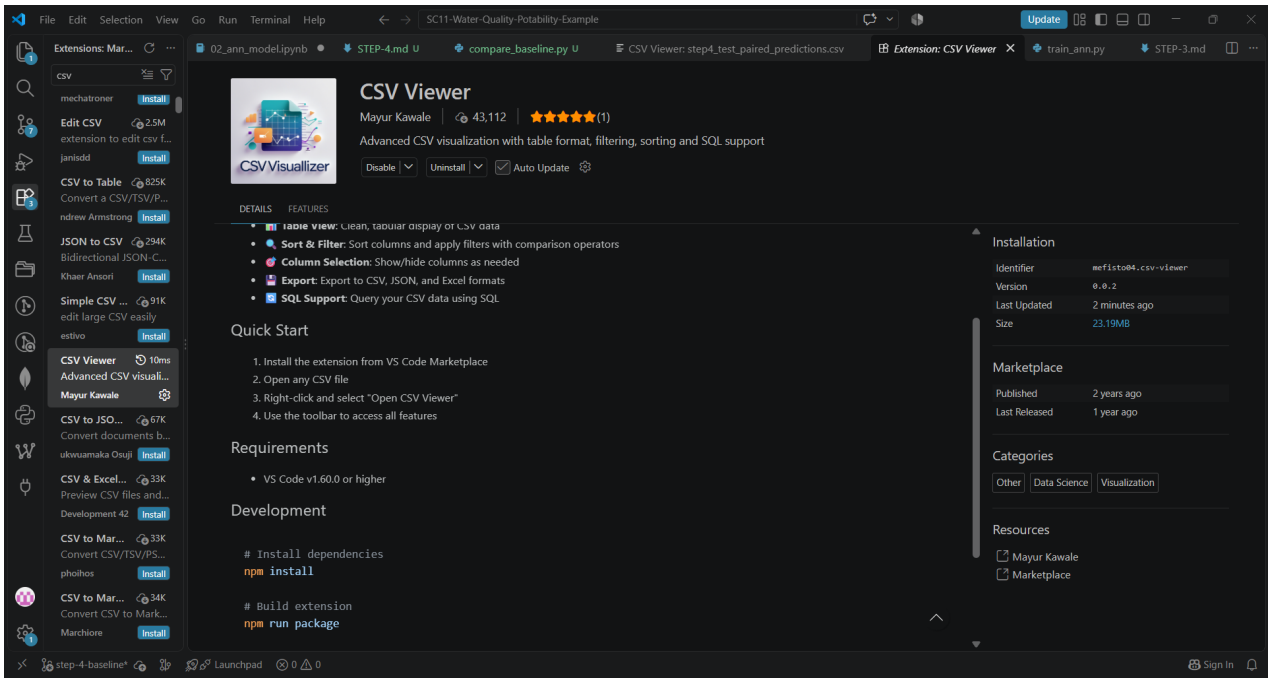
1. In the VS Code output, find validation and test rows for Majority baseline and Step 3 ANN.
2. For the optional table view, click the **Extensions** icon (four squares) on the left. Search for **CSV Viewer** by **Mayur Kawale**, open its page and click **Install**. In the screenshot the extension is already installed, so VS Code shows **Disable** and **Uninstall** instead.
3. In Explorer, expand **data** and **reports**. Click **step4_baseline_metrics.csv** once to open the ordinary comma-separated text. Confirm its first row lists split, method, rows and metrics.
4. Now **right-click the CSV filename in Explorer** (not the text inside the editor). Choose **Open CSV Viewer** from the menu. Wait for the table tab to open.
5. Read the validation and test rows for both methods. Then open **step4_comparison.csv** normally, right-click its filename in Explorer, and choose **Open CSV Viewer** again.
6. The viewer is only for easier reading; it does not create or change the CSV. If you cannot install it, the plain CSV and script output are equally valid evidence.
7. For the SC11 example only, test accuracy is 0.5000 for baseline and 0.6276 for ANN; the difference is 0.1276 score points.
8. Notice baseline precision, recall and F1 for potable class are zero because it predicts only 0. Explain this limitation.

What to check: Report score differences as points; do not call 0.1276 a 12.76 percent relative improvement without defining that calculation.

If something goes wrong: If your outputs differ, check whether your Step 2/3 data or predictions changed. Report your actual results, not this example's numbers.

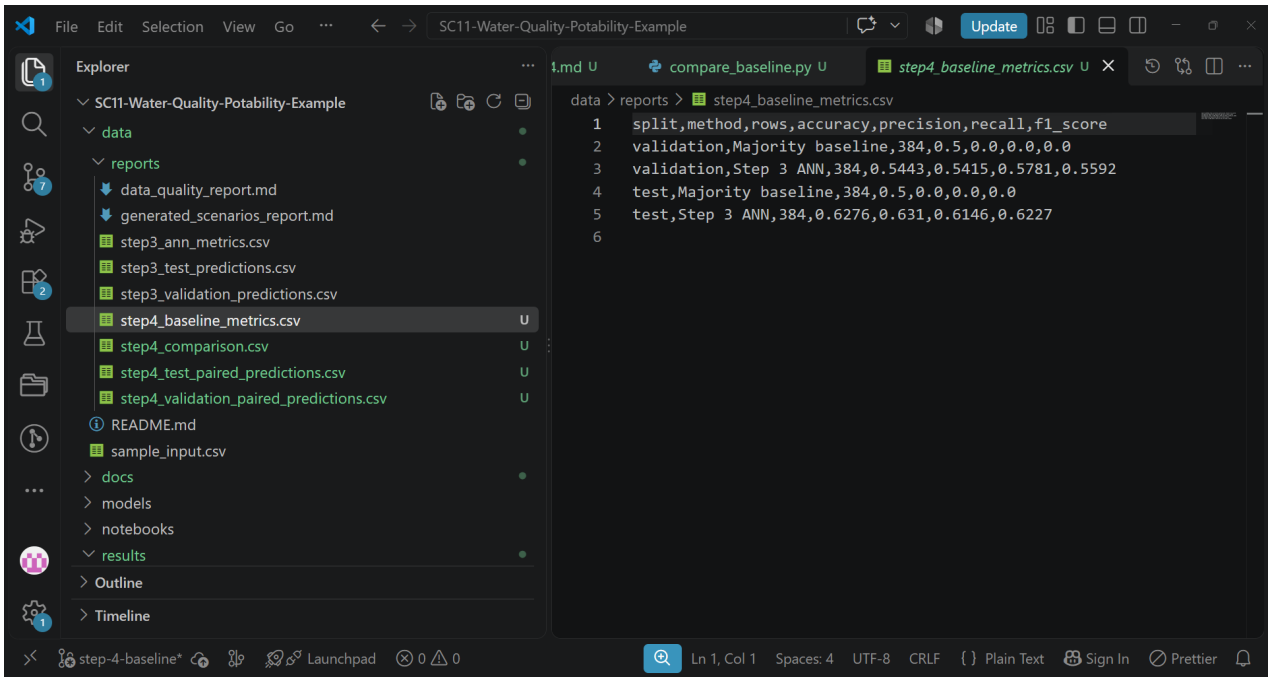
Action 8 - SC11 worked example

Find CSV Viewer by Mayur Kawale in Extensions; this screenshot shows it already installed



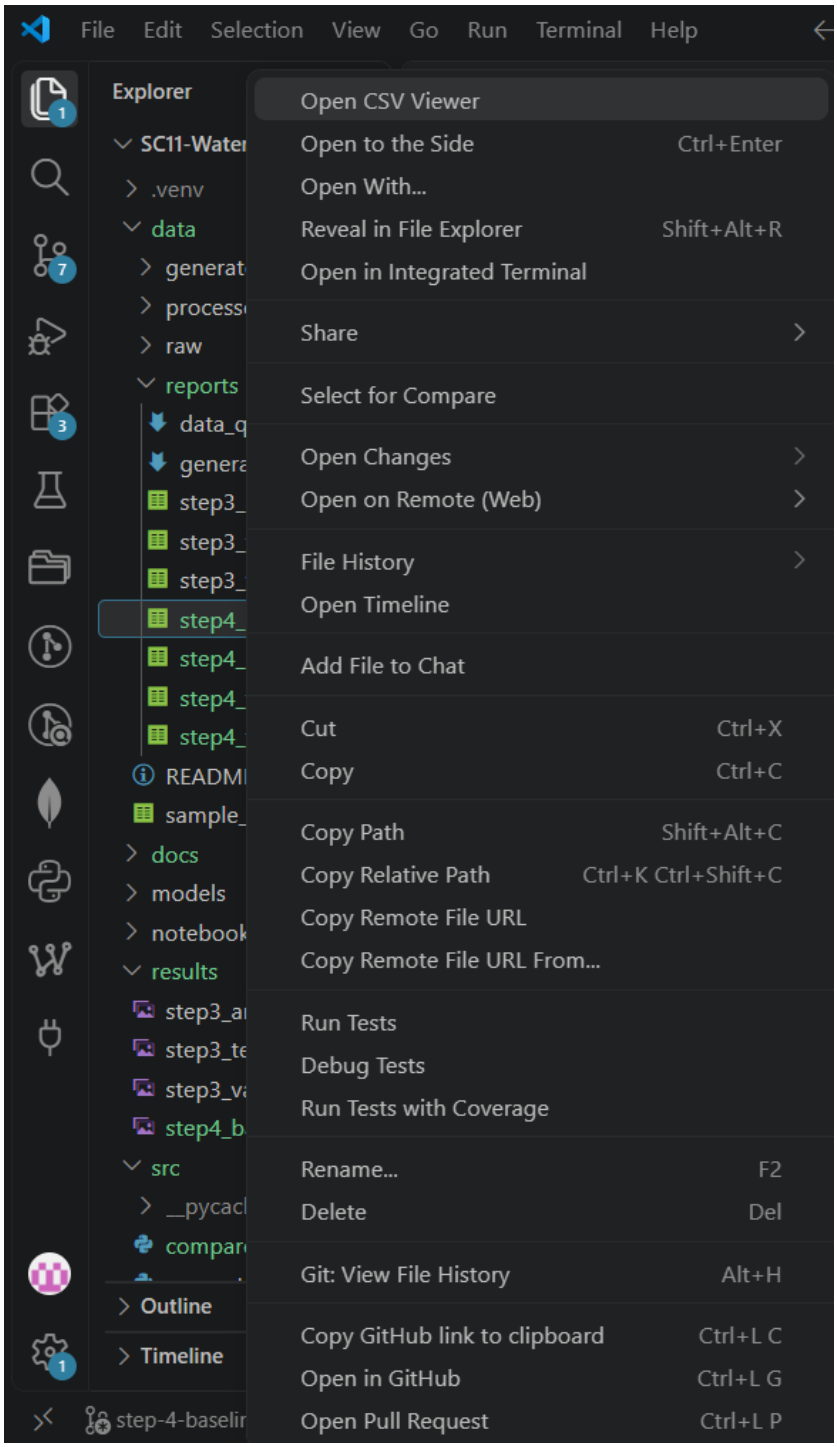
Action 8 - SC11 worked example (continued)

First open step4_baseline_metrics.csv as a regular CSV file



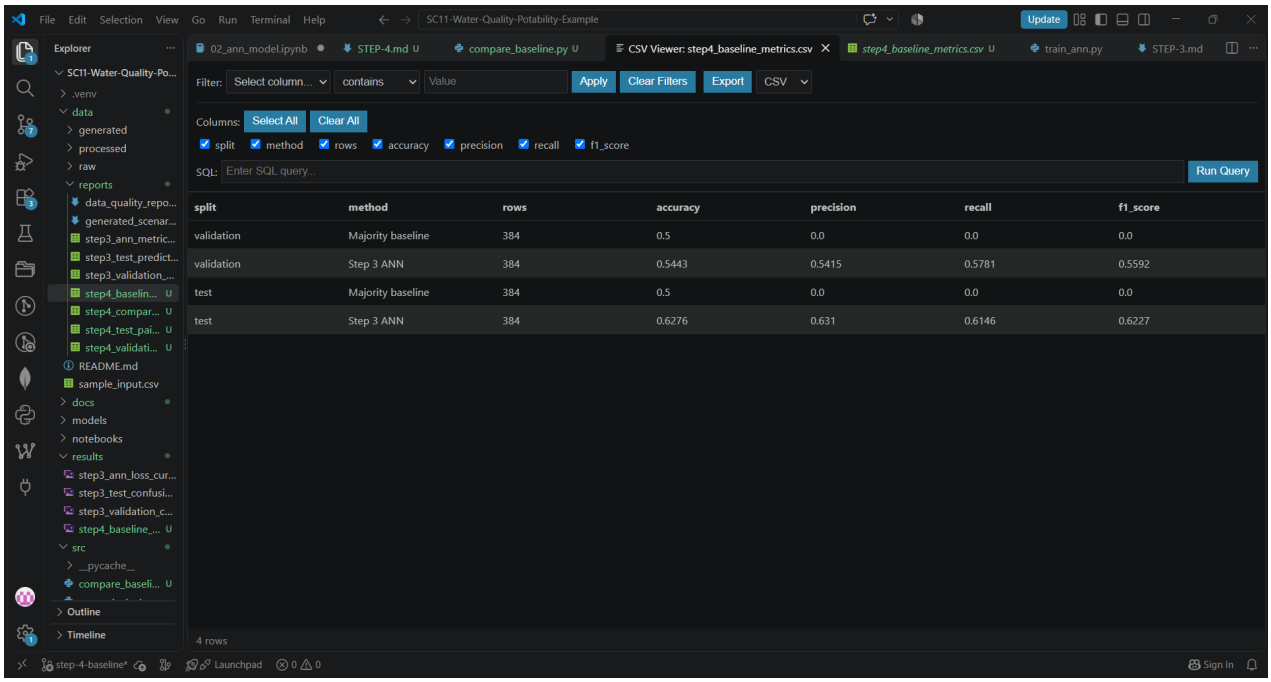
Action 8 - SC11 worked example (continued)

Right-click that CSV filename in Explorer and choose Open CSV Viewer



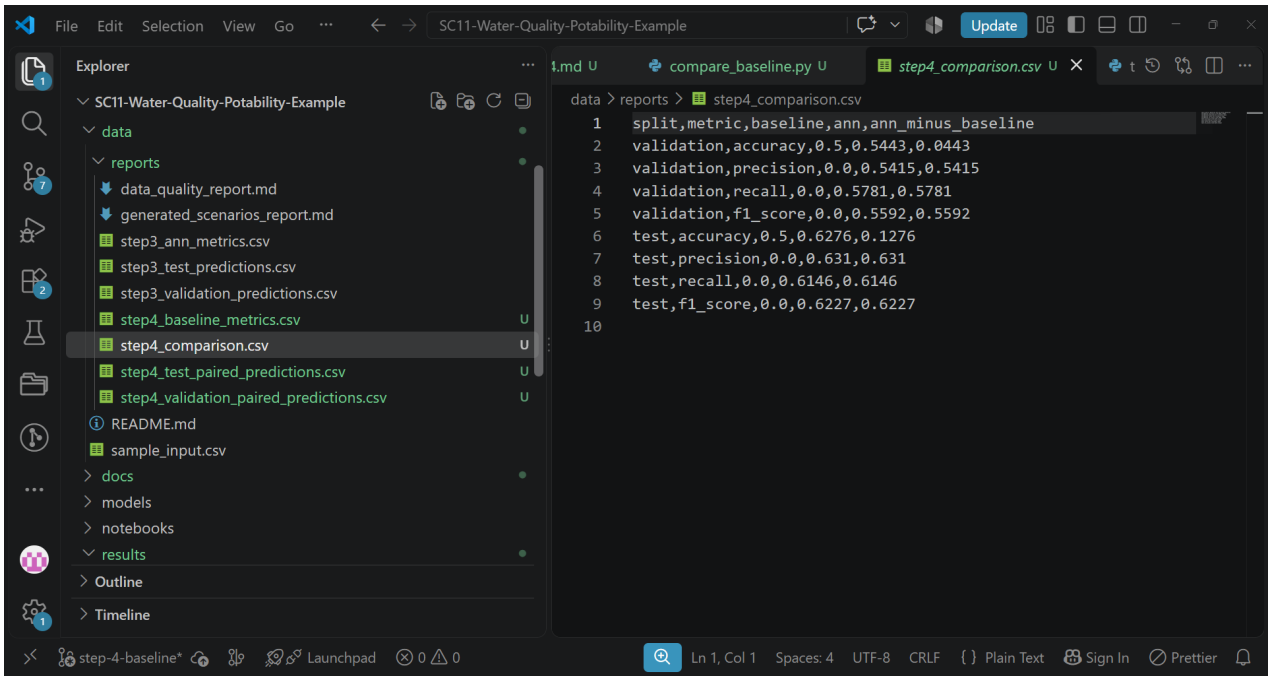
Action 8 - SC11 worked example (continued)

Read the metrics in the CSV Viewer table



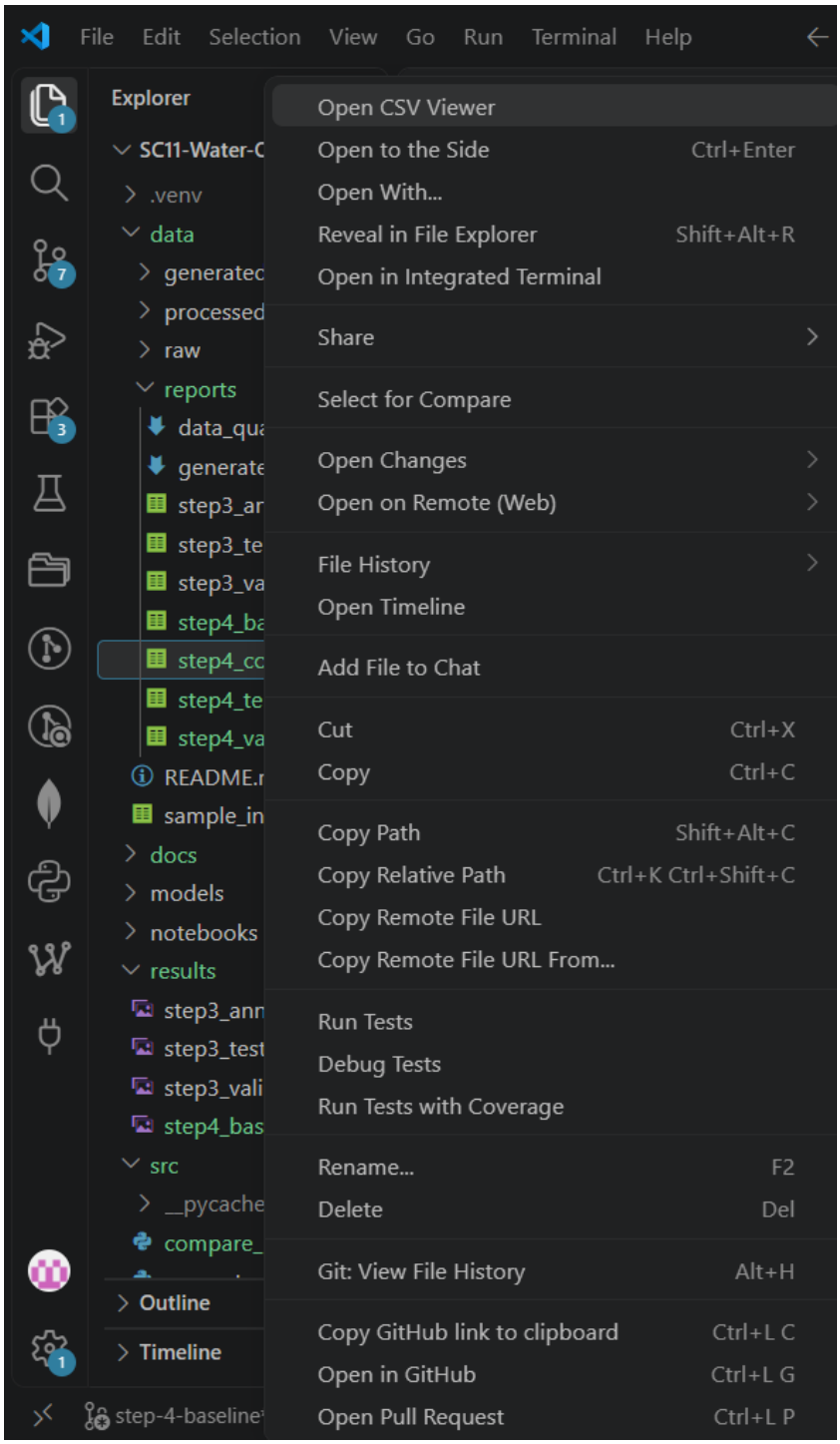
Action 8 - SC11 worked example (continued)

Open step4_comparison.csv as a regular CSV file



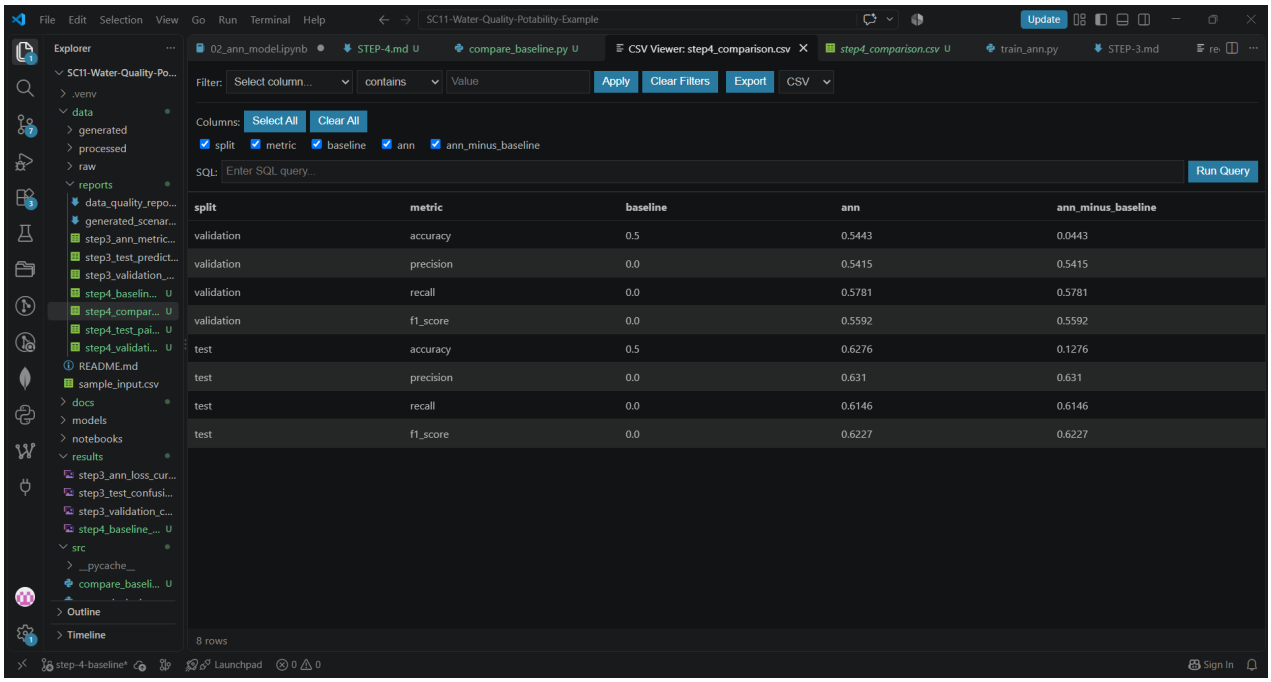
Action 8 - SC11 worked example (continued)

Right-click the comparison CSV filename and choose Open CSV Viewer



Action 8 - SC11 worked example (continued)

Read the score differences in the CSV Viewer table



Action 9: Inspect paired predictions

Why: The paired CSV proves that the two methods were scored against the same original labels.

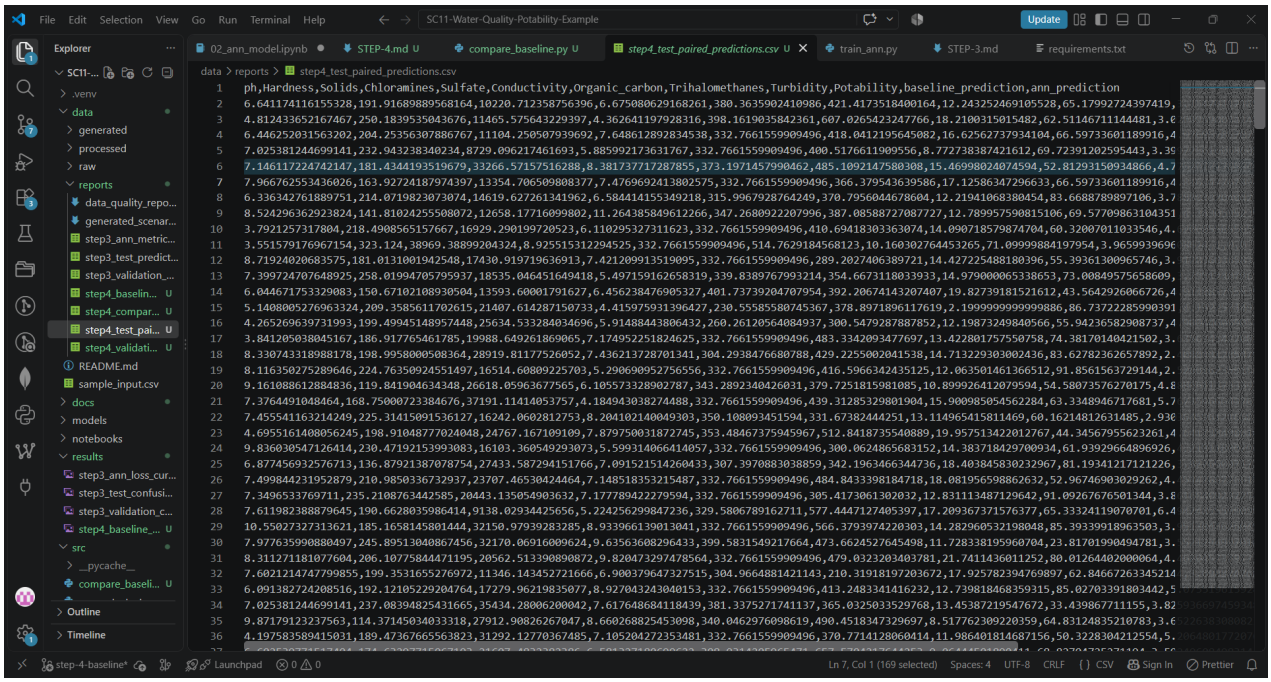
1. In Explorer, click **data/reports/step4_test_paired_predictions.csv** once. The ordinary CSV text may run far across the screen; this is normal.
2. Right-click that **filename in Explorer** and choose **Open CSV Viewer**, as in Action 8. The viewer is optional; you can also scroll across the plain CSV.
3. In the table find **Potability**, **baseline_prediction** and **ann_prediction**. These are the original label and the two predictions for the same row.
4. Check several rows, including cases where the two methods disagree.
5. Do not change true labels or manually edit predictions to improve a score.

What to check: The test file has 384 rows for the current SC11 example.

If something goes wrong: If the paired CSV is missing, rerun the script and look for an error above STEP 4 COMPLETED.

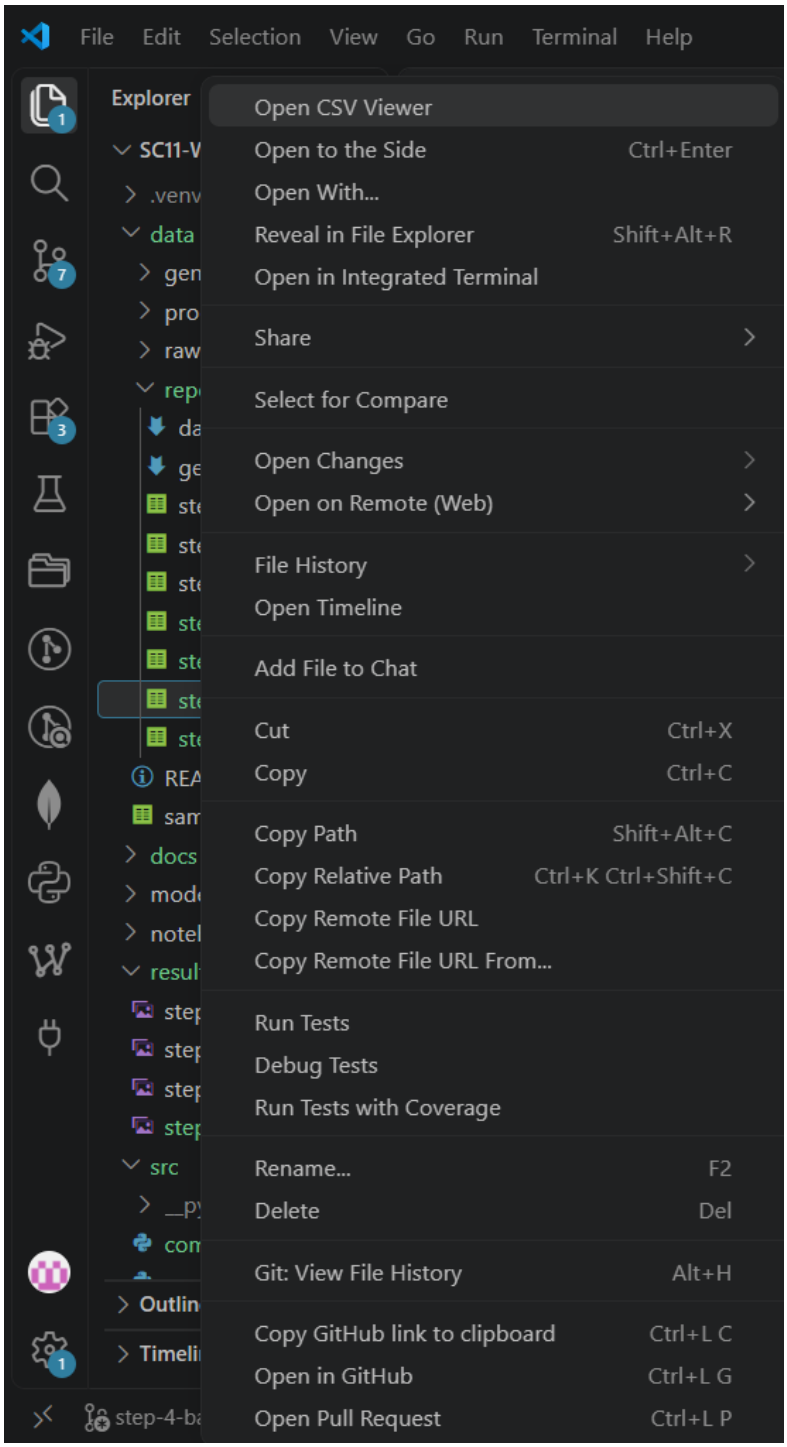
Action 9 - SC11 worked example

First open step4_test_paired_predictions.csv as a regular CSV file



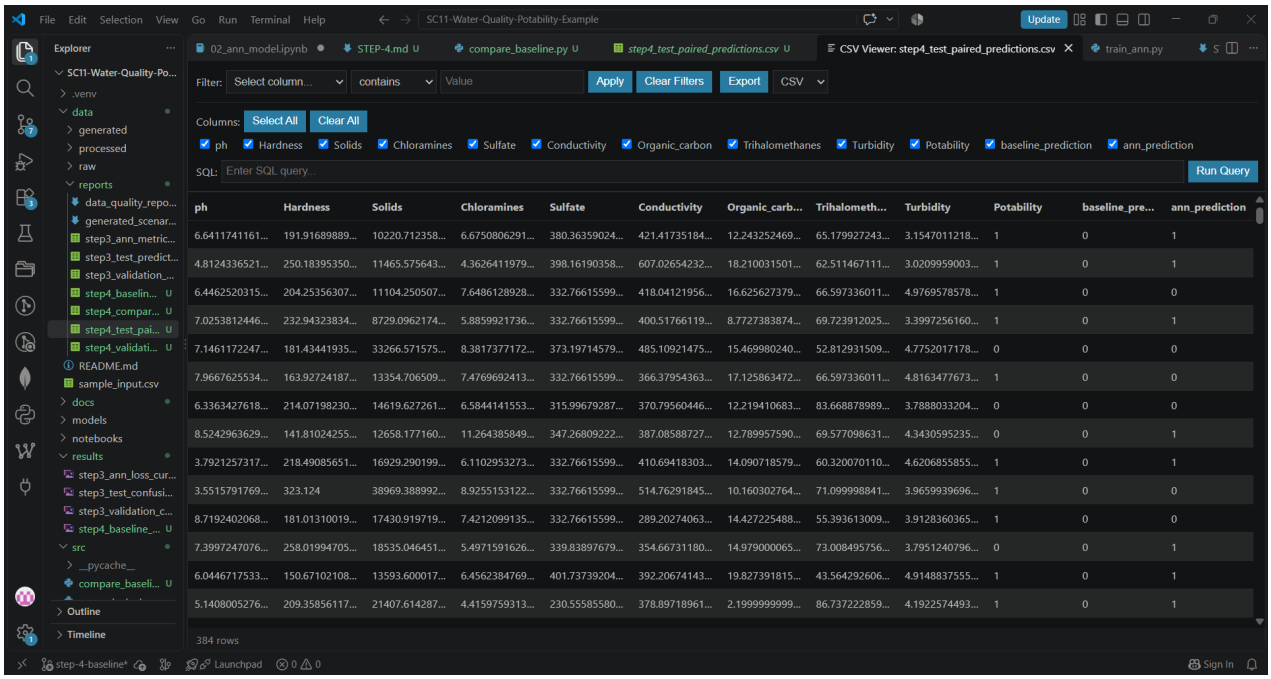
Action 9 - SC11 worked example (continued)

Right-click the paired-predictions CSV filename and choose Open CSV Viewer



Action 9 - SC11 worked example (continued)

Inspect original labels and both predictions on the same rows



Action 10: Inspect the comparison chart

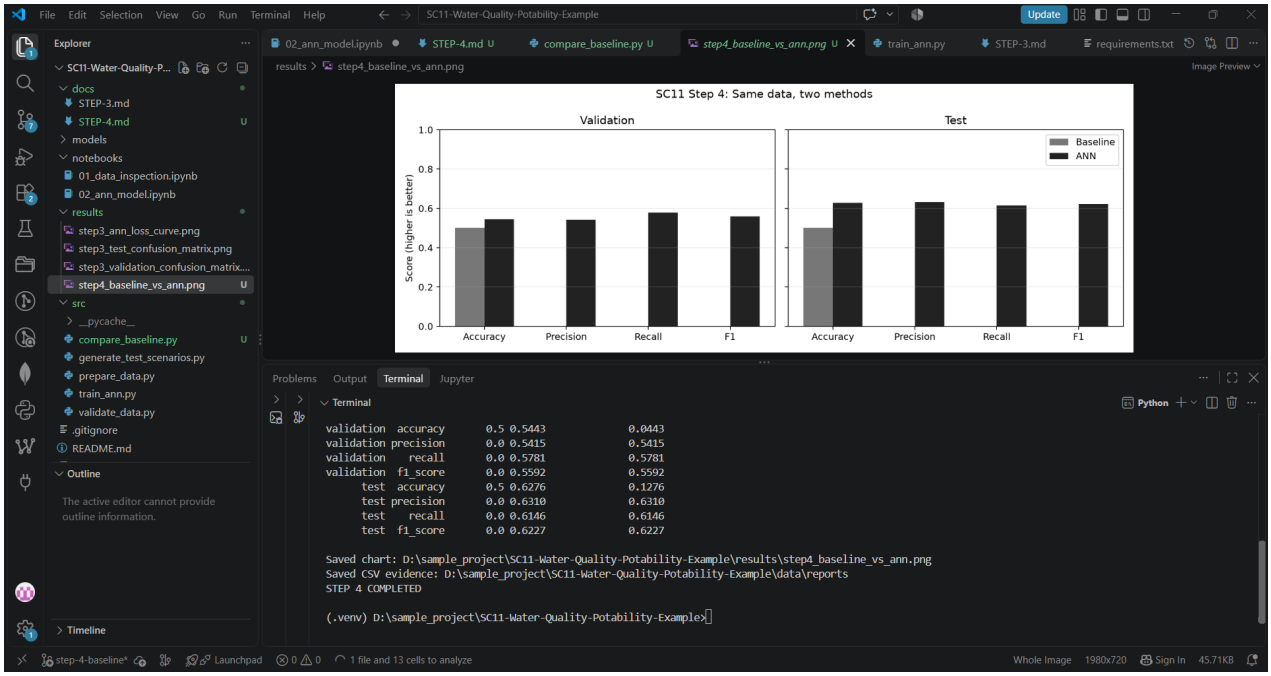
Why: A figure helps readers see strengths and weaknesses across several measures.

1. In Explorer, open results/step4_baseline_vs_ann.png.
2. Check both panels: validation and test.
3. Confirm the legend distinguishes baseline from ANN and the axis is labelled Score.
4. Discuss the chart in docs/STEP-4.md instead of inserting an unexplained image.

What to check: The graph compares the same four measures on both held-out splits.

If something goes wrong: If the graph is blank, inspect the CSV metrics and rerun the Python file.

Comparison chart and completed script output



Action 11: Create and run the notebook

Why: An executed notebook gives reviewers visible calculations and a few example predictions.

1. In Explorer, right-click **notebooks**, choose **New File**, and choose a suitable .ipynb name (SC11 uses **03_baseline_comparison.ipynb**.)
2. Appendix D shows SC11's Markdown and code cell sequence. Use VS Code's **+ Markdown** and **+ Code** buttons to create your own analysis; do not paste notebook JSON into one code cell.
3. Open the notebook in VS Code, select the same Python kernel used in Step 3, then click **Run All**.
4. Read each cell's output: project path, class counts, metrics, comparison, chart and paired rows.
5. Save the notebook after execution so outputs remain in the committed .ipynb file.

SC11 code excerpt - adapt the idea to your own project

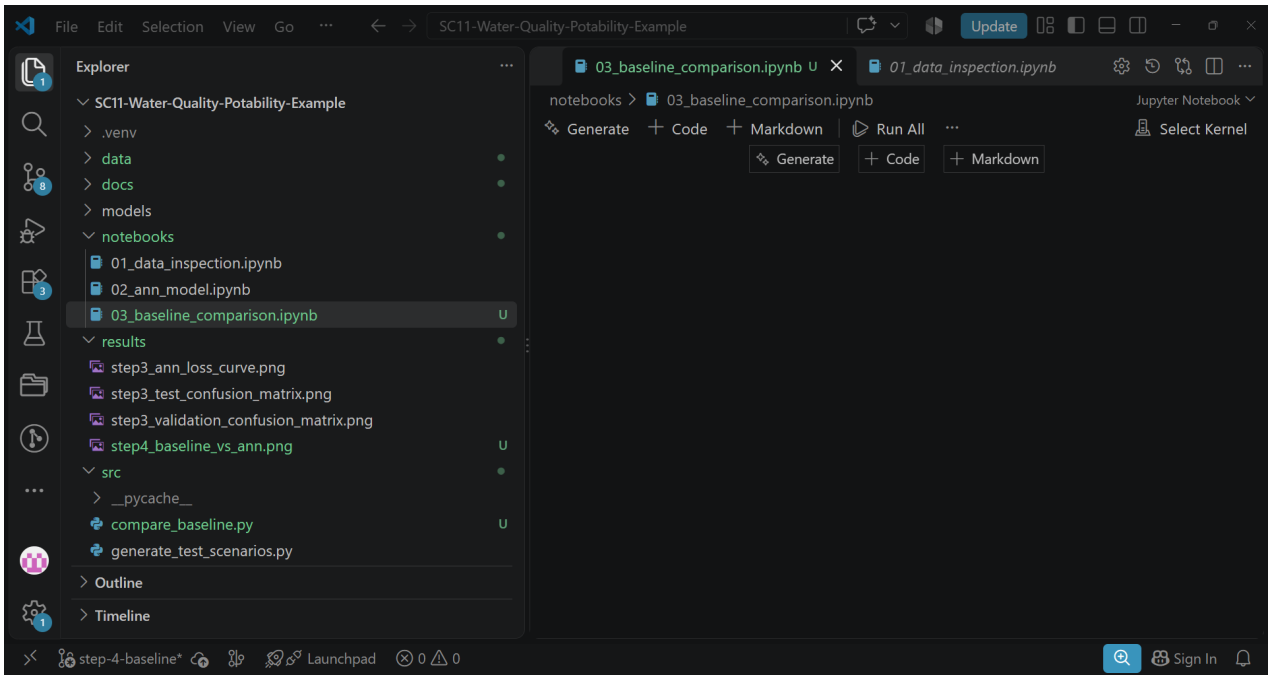
```
# SC11 notebook idea: call a repeatable script
from src.compare_baseline import main
main()
# Adapt the module and explain its output below.
```

What to check: Every code cell has a successful execution count and no red error box.

If something goes wrong: If the notebook cannot find your script, open the repository folder itself in VS Code and rerun from the first cell.

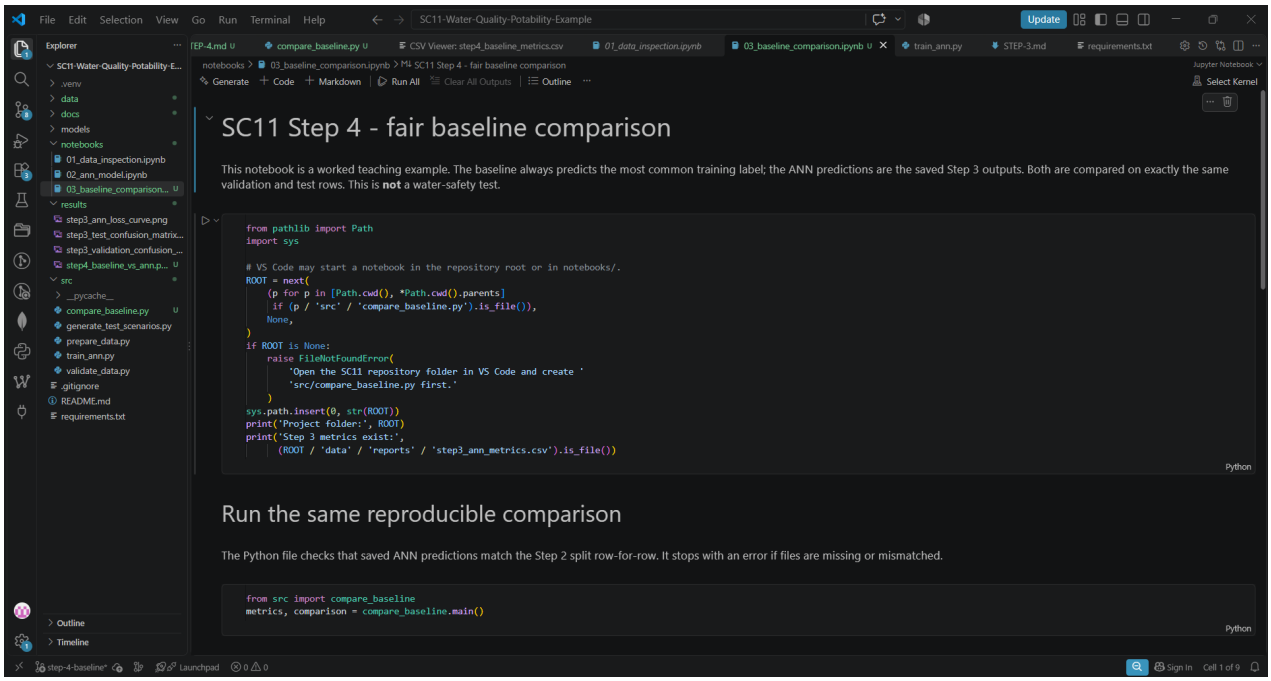
Action 11 - SC11 worked example

Create 03_baseline_comparison.ipynb in notebooks



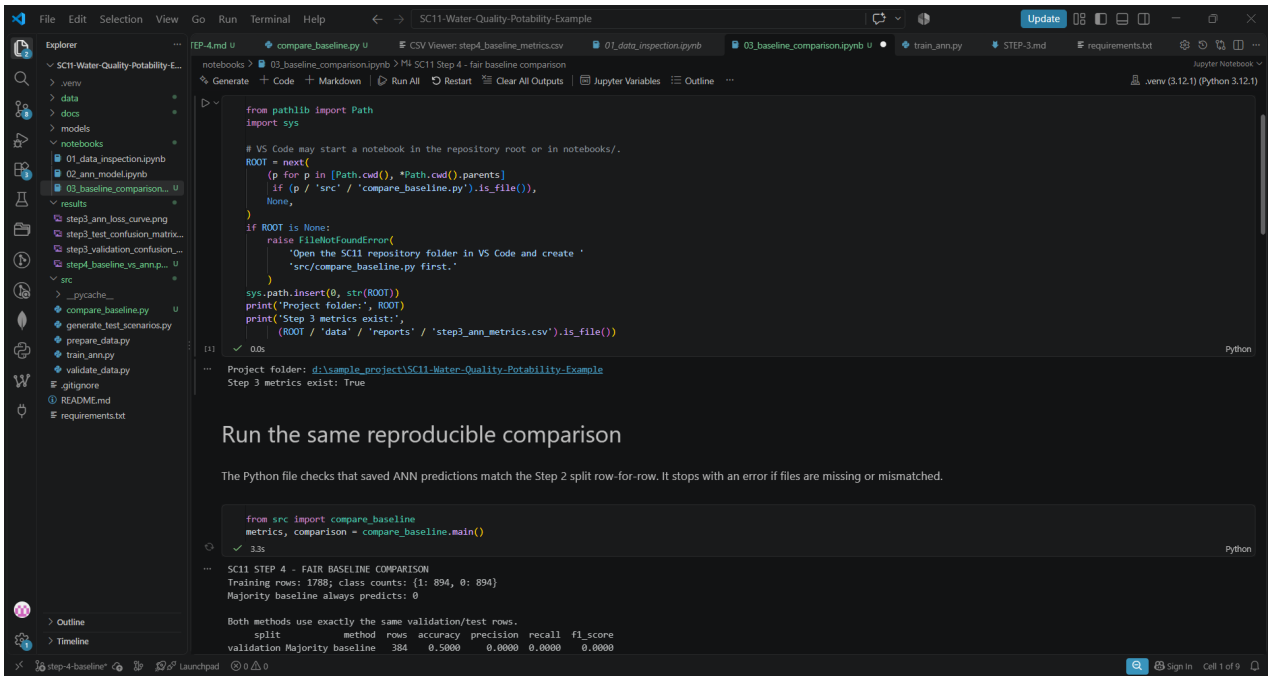
Action 11 - SC11 worked example (continued)

Notebook cells before execution



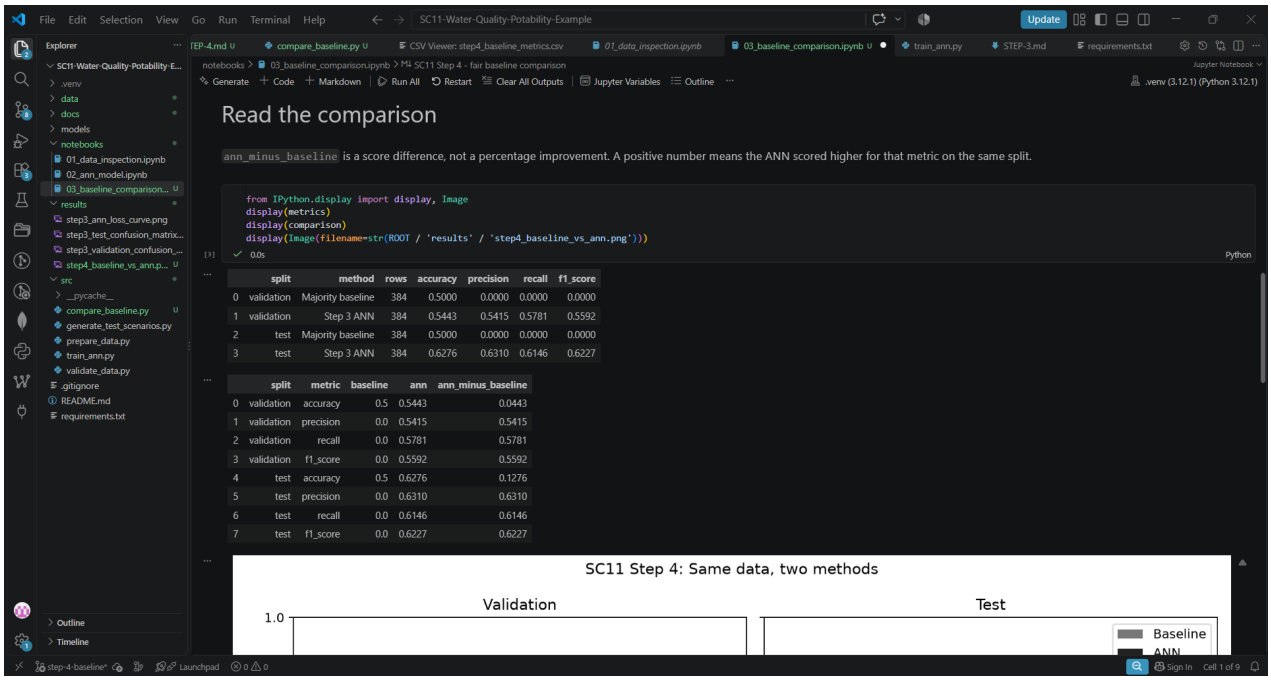
Action 11 - SC11 worked example (continued)

Executed reproducible comparison



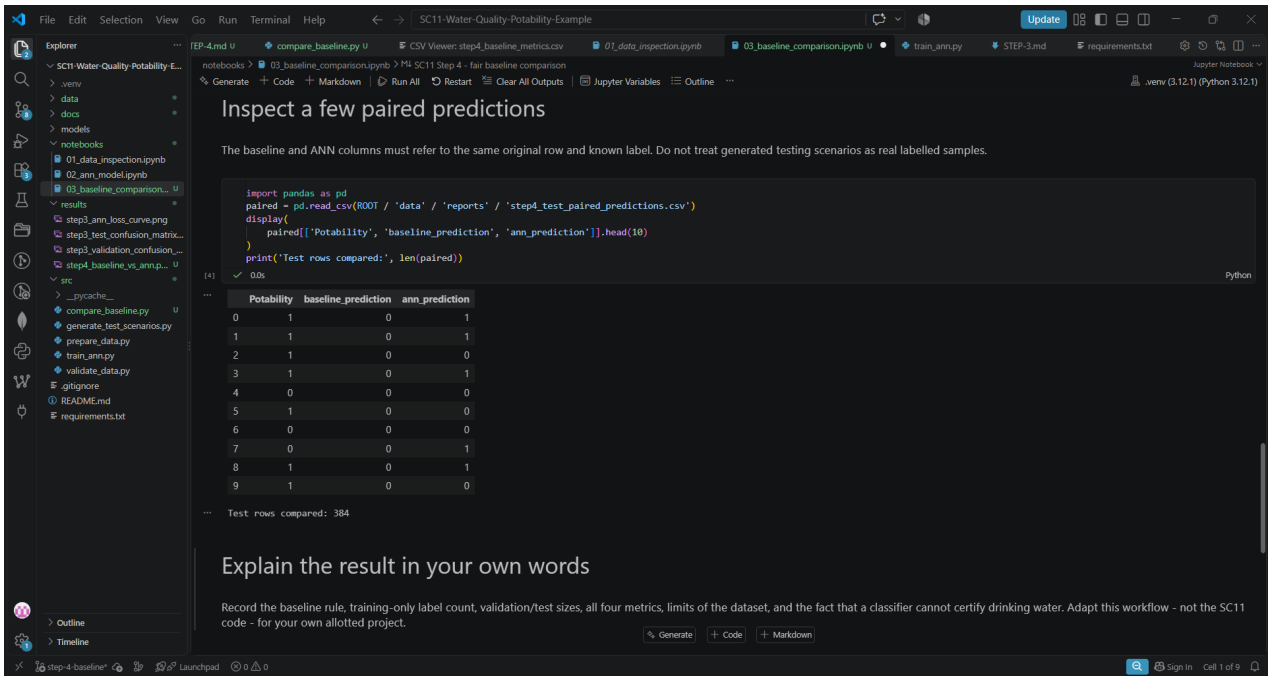
Action 11 - SC11 worked example (continued)

Rendered metrics, differences and chart



Action 11 - SC11 worked example (continued)

Paired predictions displayed in the notebook; save before commit



Action 12: Complete the plain-language report

Why: Reviewers need to know what the numbers mean and what they do not prove.

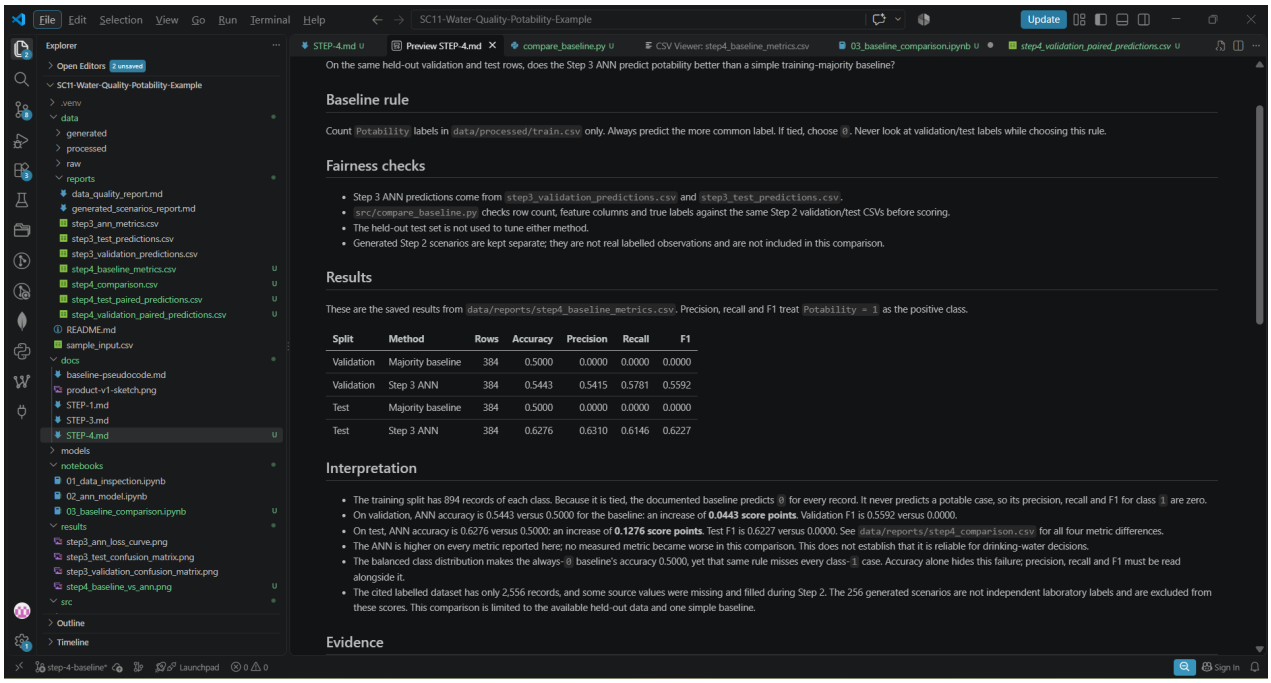
1. In VS Code Explorer, expand **docs** and click the existing **STEP-4.md**. No new Python file is needed for this action.
2. Compare its Results table with **data/reports/step4_baseline_metrics.csv**. The SC11 worked report in Appendix B now contains the measured 384-row validation and test results; other teams must enter their own values.
3. Read the **Interpretation** and **Safety and scope** sections. For SC11, test accuracy increased by 0.1276 score points, while an always-0 baseline missed every potable case. This is a measured example, not a claim of drinking-water safety.
4. For an allocated student team, replace the Team contributions section with each member's real name and specific work. SC11 is an instructor example, so it honestly says there are no student contributors.
5. Save the Markdown file. If you prefer a formatted view, right-click the open STEP-4.md editor and choose **Open Preview** (or click the preview icon); check the table, interpretation, evidence and limitations are readable. Do not change the CSV results just to match the prose.

What to check: All four rows and values must match the saved CSV; SC11's example must not pretend to have student contributors.

If something goes wrong: If the table is still blank, reopen and save STEP-4.md, then compare it against the CSVs. Student teams must not copy SC11 values.

Action 12 - SC11 worked example

Completed Results table in the STEP-4.md preview



On the same held-out validation and test rows, does the Step 3 ANN predict potability better than a simple training-majority baseline?

Baseline rule

Count `Potability` labels in `data/processed/train.csv` only. Always predict the more common label. If tied, choose `0`. Never look at validation/test labels while choosing this rule.

Fairness checks

- Step 3 ANN predictions come from `step3_validation_predictions.csv` and `step3_test_predictions.csv`.
- `src/compare_baseline.py` checks row count, feature columns and true labels against the same Step 2 validation/test CSVs before scoring.
- The held-out test set is not used to tune either method.
- Generated Step 2 scenarios are kept separate; they are not real labelled observations and are not included in this comparison.

Results

These are the saved results from `data/reports/step4_baseline_metrics.csv`. Precision, recall and F1 treat `Potability = 1` as the positive class.

Split	Method	Rows	Accuracy	Precision	Recall	F1
Validation	Majority baseline	384	0.5000	0.0000	0.0000	0.0000
Validation	Step 3 ANN	384	0.5443	0.5415	0.5781	0.5592
Test	Majority baseline	384	0.5000	0.0000	0.0000	0.0000
Test	Step 3 ANN	384	0.6276	0.6310	0.6146	0.6227

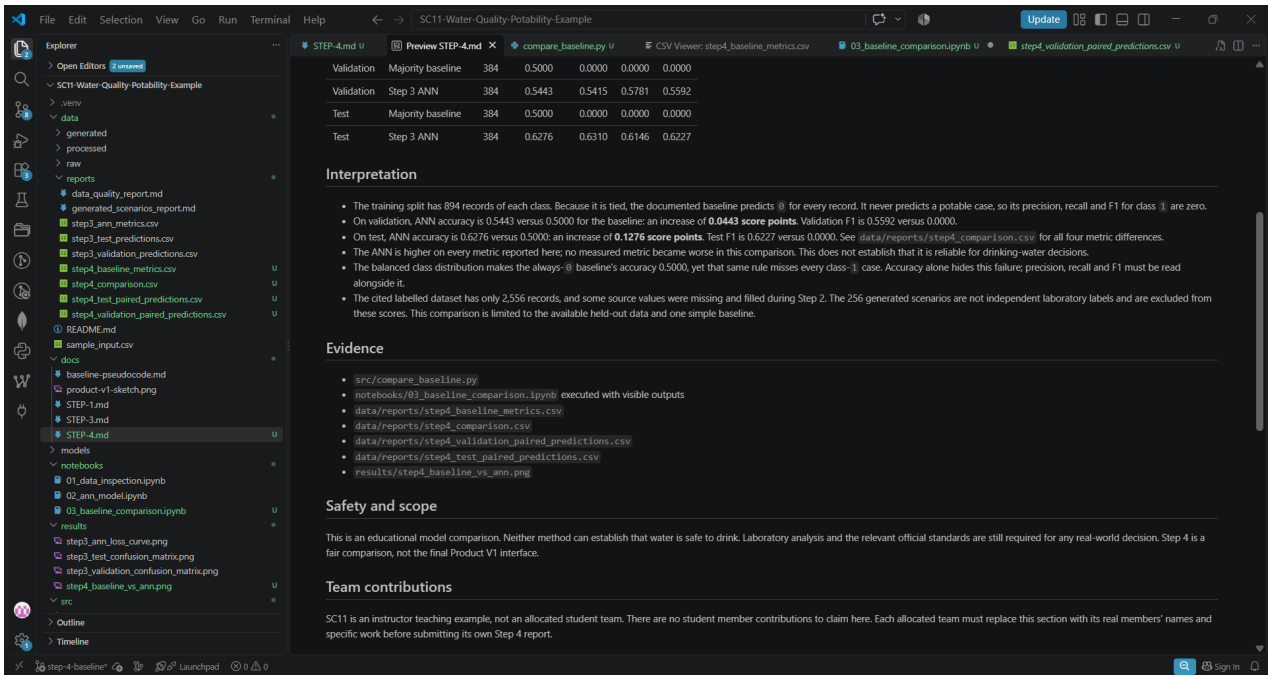
Interpretation

- The training split has 894 records of each class. Because it is tied, the documented baseline predicts `0` for every record. It never predicts a potable case, so its precision, recall and F1 for class `1` are zero.
- On validation, ANN accuracy is 0.5443 versus 0.5000 for the baseline: an increase of **0.0443 score points**. Validation F1 is 0.5592 versus 0.0000.
- On test, ANN accuracy is 0.6276 versus 0.5000: an increase of **0.1276 score points**. Test F1 is 0.6227 versus 0.0000. See `data/reports/step4_comparison.csv` for all four metric differences.
- The ANN is higher on every metric reported here; no measured metric became worse in this comparison. This does not establish that it is reliable for drinking-water decisions.
- The balanced class distribution makes the always-`0` baseline's accuracy 0.5000, yet that same rule misses every class-`1` case. Accuracy alone hides this failure; precision, recall and F1 must be read alongside it.
- The cited labelled dataset has only 2,556 records, and some source values were missing and filled during Step 2. The 256 generated scenarios are not independent laboratory labels and are excluded from these scores. This comparison is limited to the available held-out data and one simple baseline.

Evidence

Action 12 - SC11 worked example (continued)

Interpretation, evidence, safety and scope, and the honest SC11 contribution note



Action 13: Stage and commit the Step 4 work

Why: A commit is a named checkpoint of the files you intentionally include.

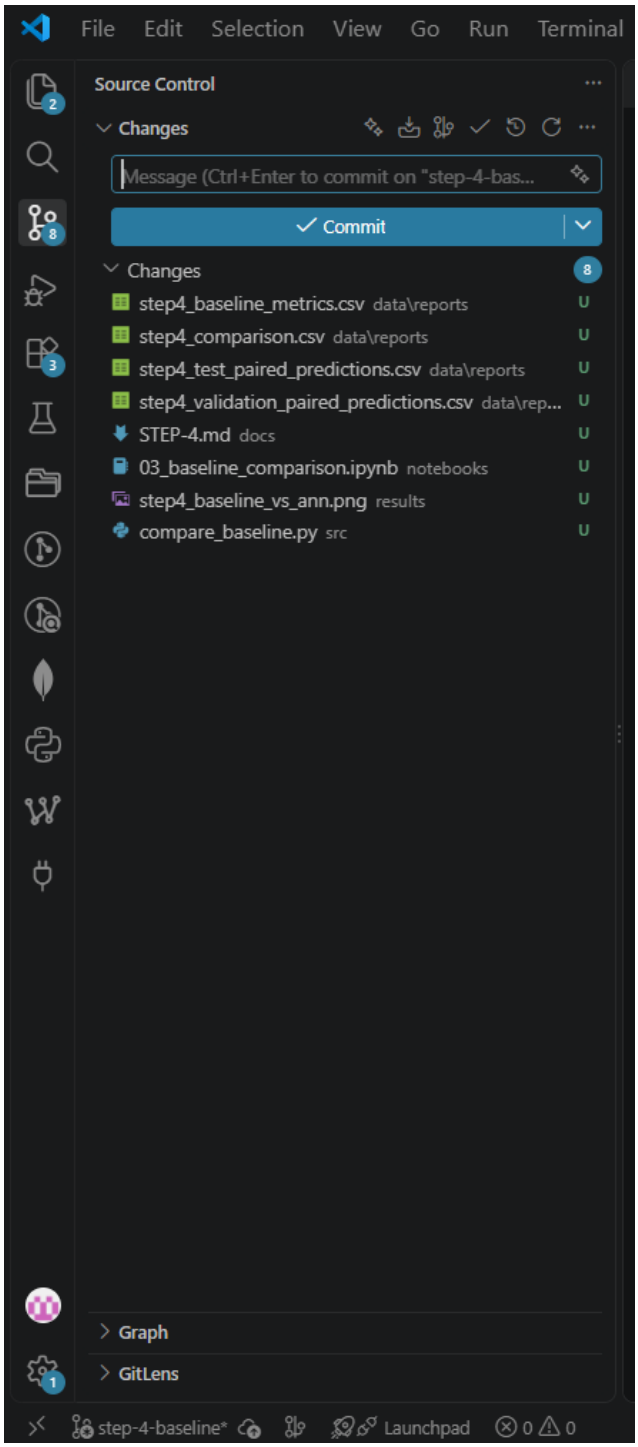
1. Click Source Control in VS Code and review the changed-file list.
2. Stage the script, executed notebook, completed STEP-4.md, new CSVs and chart. Do not stage private or temporary files.
3. Enter a meaningful message such as **Add fair SC11 baseline comparison**.
4. Click Commit. Confirm the changed-file list becomes clean or shows only unrelated files.

What to check: The commit contains evidence, not only a documentation claim.

If something goes wrong: If the Commit button is disabled, save the files and stage the intended changes first.

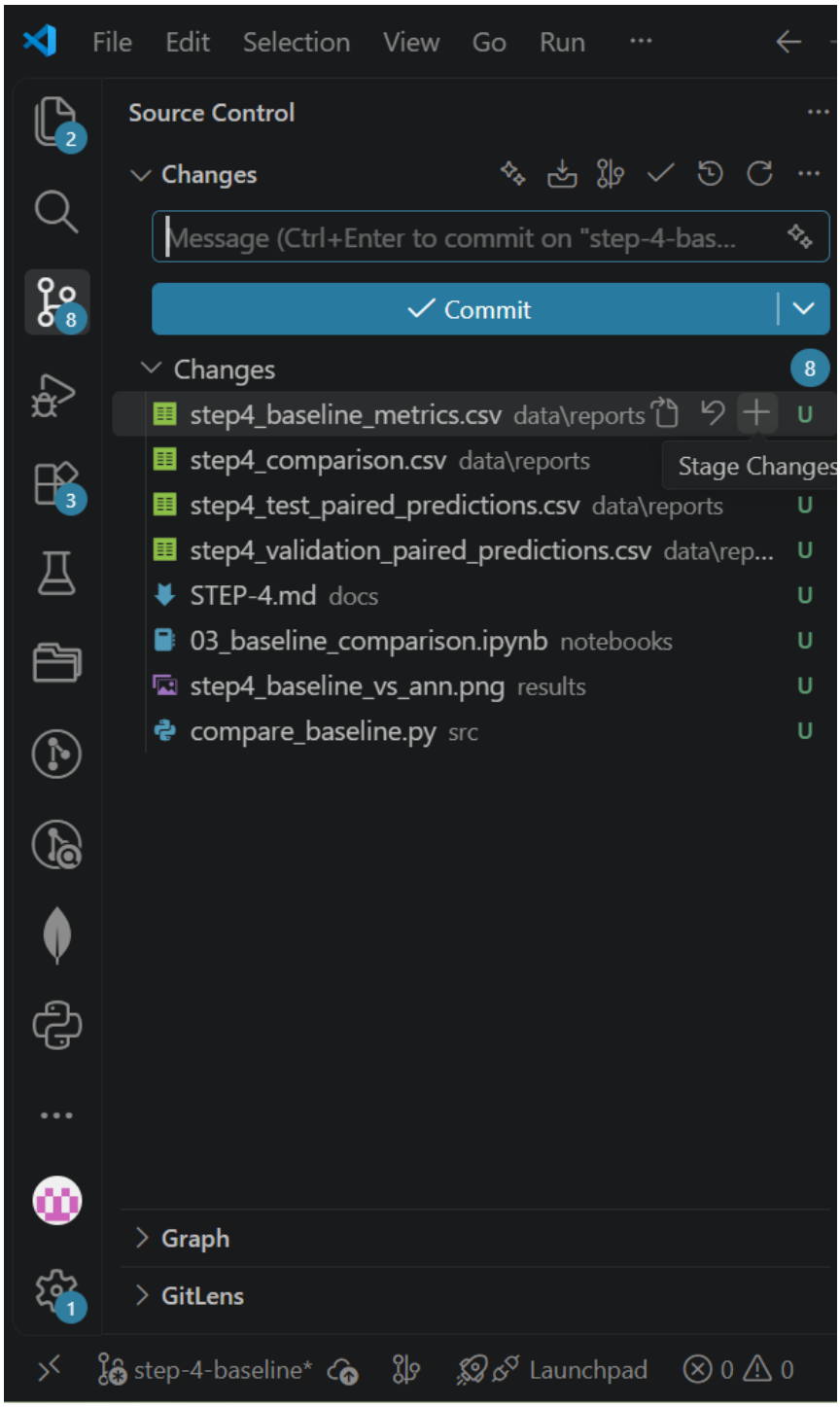
Action 13 - SC11 worked example

Review the eight new Step 4 files in Source Control



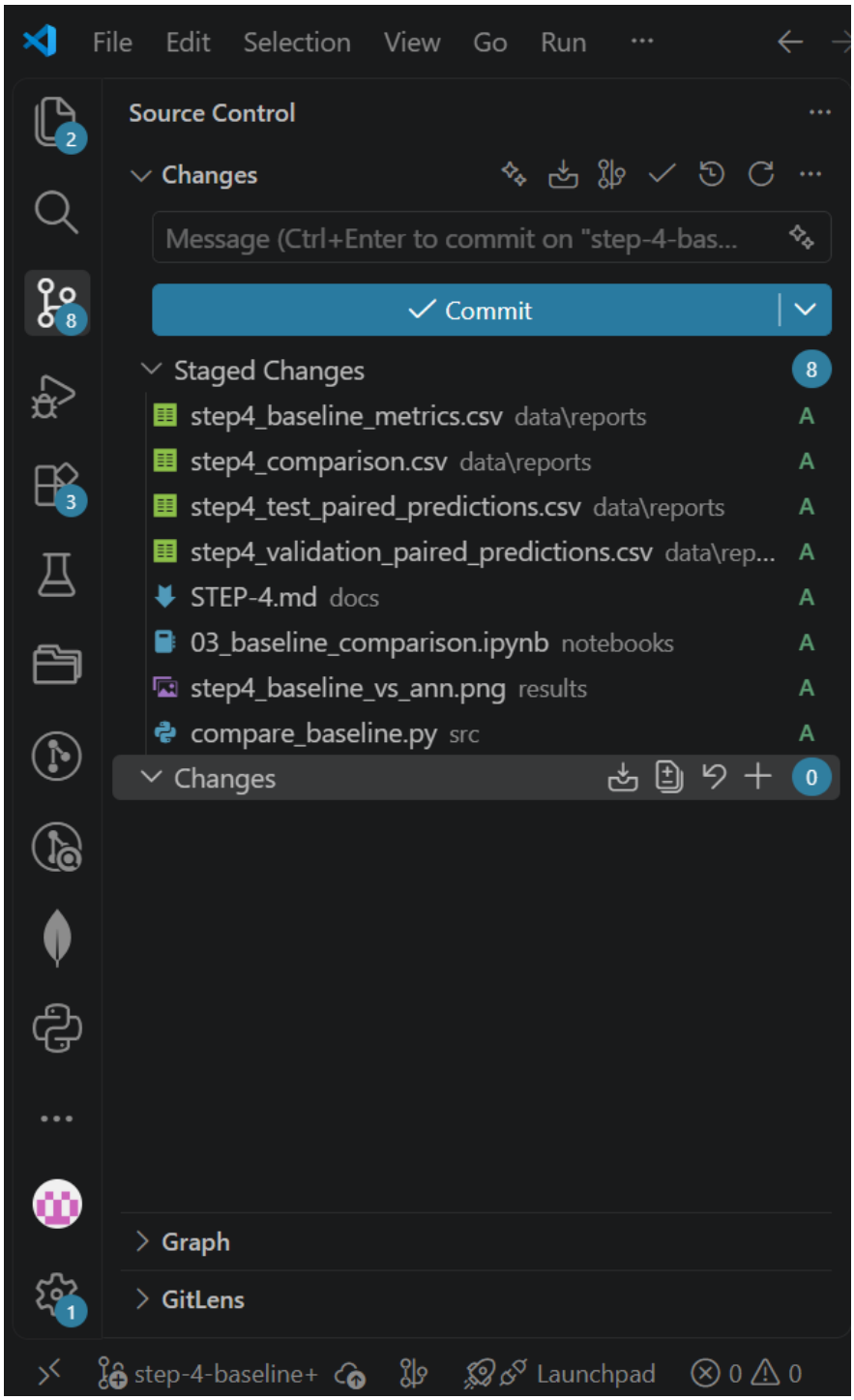
Action 13 - SC11 worked example (continued)

Use the plus control to stage the intended files



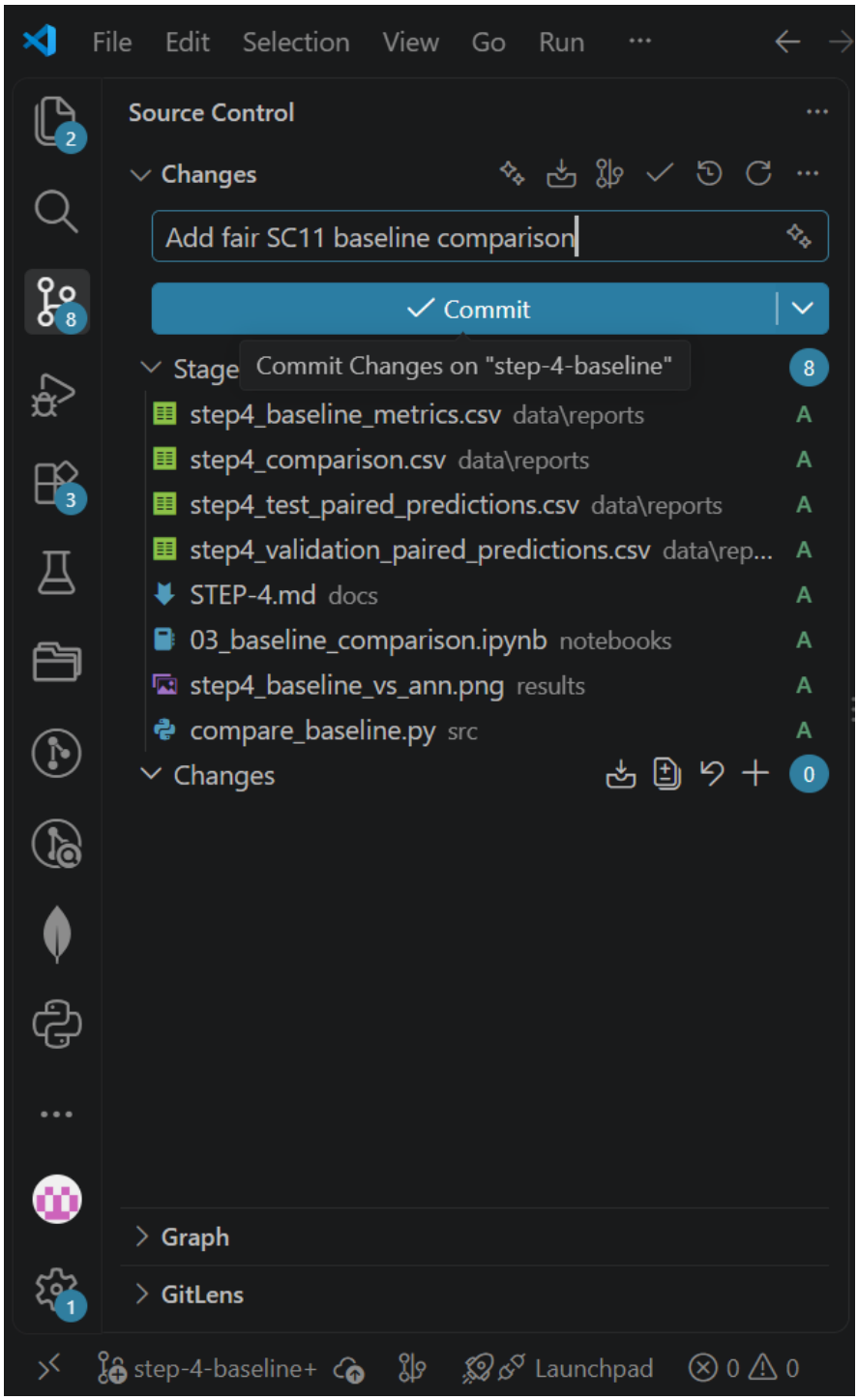
Action 13 - SC11 worked example (continued)

Confirm all eight intended files are staged



Action 13 - SC11 worked example (continued)

Enter a meaningful commit message before clicking Commit



Action 14: Sync and verify on GitHub

Why: A local commit is not yet visible to the instructor until it reaches GitHub.

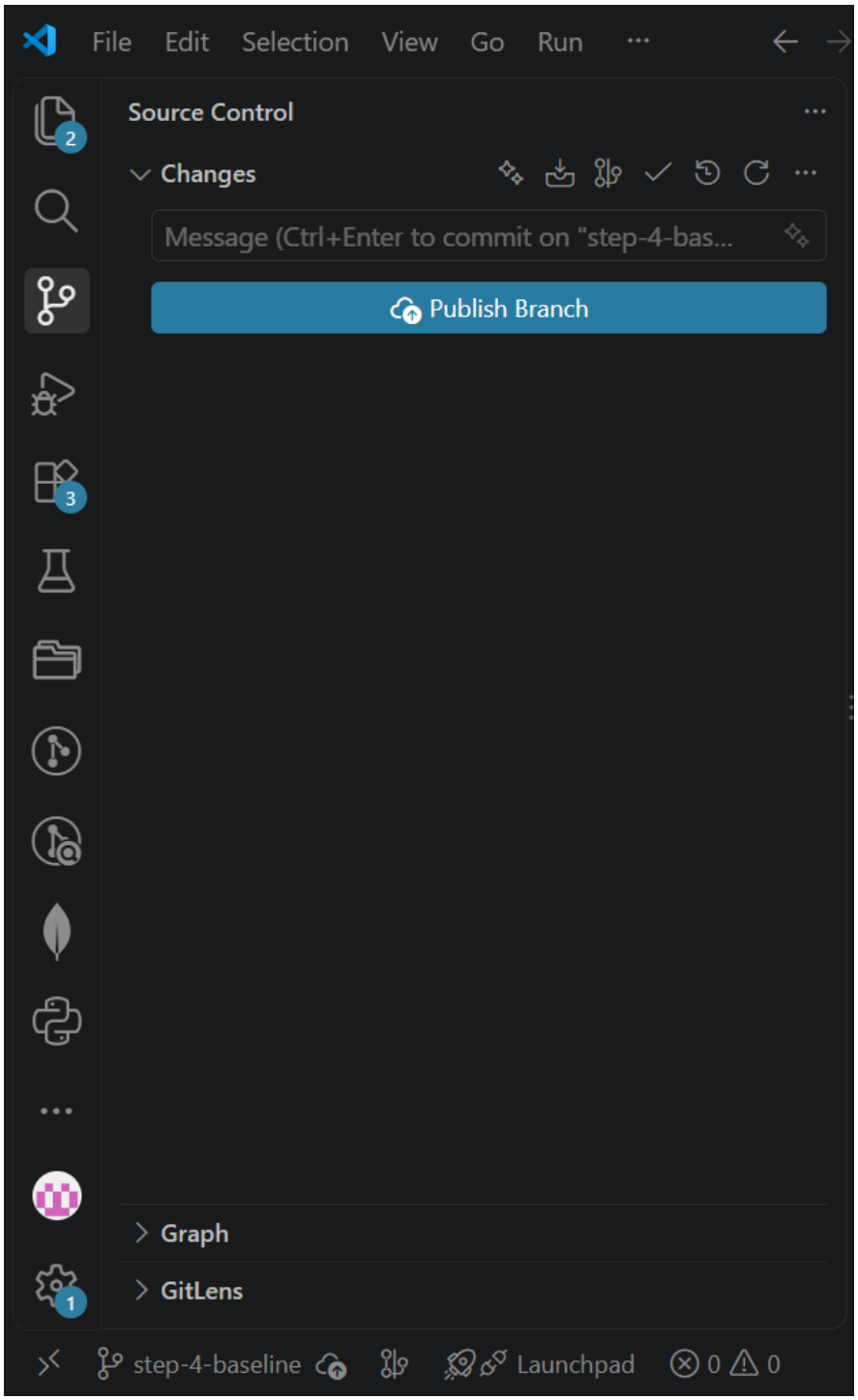
1. Click **Publish Branch** or **Sync Changes** in VS Code and wait for completion.
2. A 'Publishing Branch...' message means the upload is still running; do not treat this intermediate screen as proof of completion.
3. Open https://github.com/vipulshahi-ai/SC11-Water-Quality-Potability-Example in a browser.
4. Use the branch selector to choose step-4-baseline.
5. Check the latest commit, new files and Step 3 files on this branch. Keep main unchanged unless your team later chooses to merge.

What to check: The online branch contains both earlier Step 3 work and the new Step 4 evidence.

If something goes wrong: If GitHub shows an old commit, return to VS Code and wait for Sync/Publish to finish.

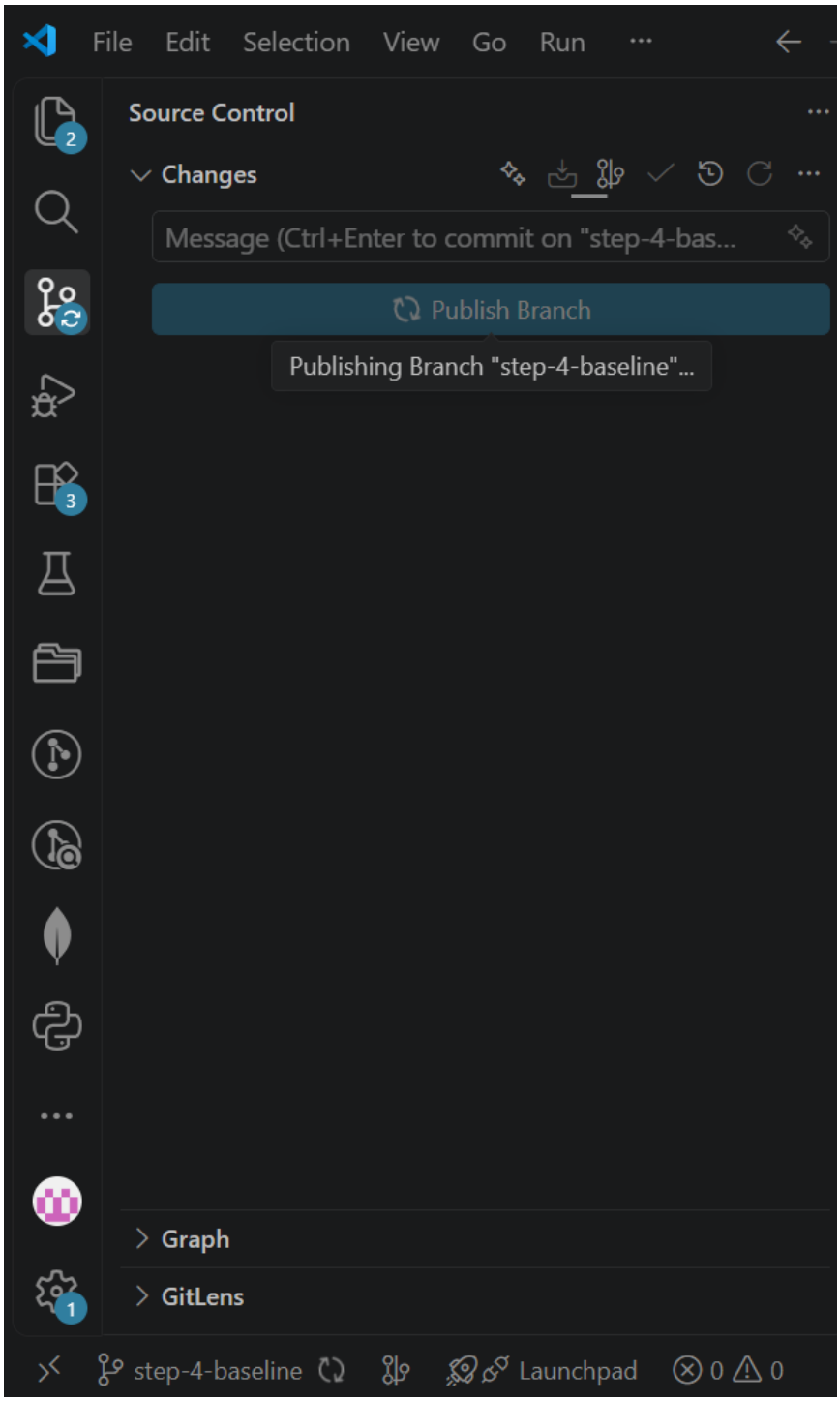
Action 14 - SC11 worked example

After the local commit, choose Publish Branch



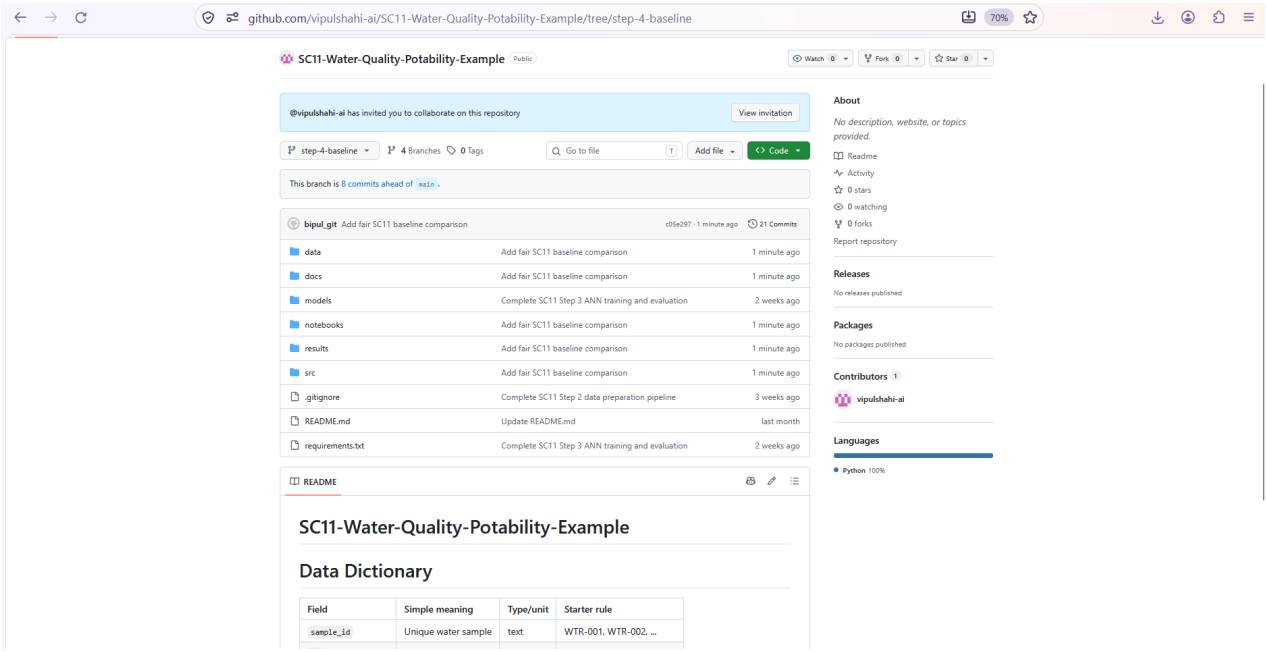
Action 14 - SC11 worked example (continued)

Publishing in progress - this is not yet proof of completion



Action 14 - SC11 worked example (continued)

Verified online: step-4-baseline shows the latest Step 4 commit and project folders



Action 15: Submit Step 4 on the student portal

Why: GitHub holds the work; the portal records the milestone submission and deadline for an allocated student team.

1. Visit https://galgotias-projects.github.io/Soft-Computing-Project-Selection/.
2. The team leader opens the dashboard with the admission number and access code already provided privately.
3. Choose Step 4. Enter the public project repository URL for your own team, not this SC11 example URL.
4. Write a short note pointing to src/compare_baseline.py, the executed notebook, docs/STEP-4.md and the comparison CSV.
5. An allocated student team submits and confirms the dashboard shows Step 4 as submitted. Keep the access code private.
6. SC11 is an instructor demonstration, not an allocated portal team: do not submit the SC11 example through a student's dashboard.

What to check: The portal records Step 4 separately from Step 3 for actual teams; do not create a false SC11 submission.

If something goes wrong: If Step 4 is locked, verify Step 3 was submitted on the same team dashboard before asking for help.

If something fails

Message or symptom	Likely cause	What to do
Missing train.csv / test.csv	Step 2 files not on this branch	Create branch from completed Step 3; sync; inspect data/processed.
Missing step3 predictions	Step 3 outputs absent	Run and save Step 3 before Step 4; do not invent scores.
Predictions do not match split	Rows reordered or different branch/data	Restore matching Step 2/3 files; keep the check enabled.
Module not found	Wrong Python interpreter	Select the interpreter that ran Step 3; use VS Code GUI.
Notebook has blank outputs	Notebook not executed or not saved	Run All, inspect output, save, then commit.
GitHub missing latest work	Commit not synced	Use Publish Branch or Sync Changes; refresh GitHub.

Safety reminder

SC11 is a demonstration. Neither a majority prediction nor ANN output can establish drinking-water safety. Use laboratory analysis and applicable official standards for real decisions.

How students adapt this to SC01-SC10

The Git, notebook, documentation and submission actions remain similar. The actual baseline, data and metric must match each allotted project. These are examples, not mandatory algorithms.

Project	Simple baseline idea	Fair measure
SC01 Climate	Fixed setpoint/threshold	Comfort and energy
SC02 Washing	Fixed wash-cycle lookup	Water, energy, time
SC03 Academic	Majority/attendance threshold	Recall, precision, errors
SC04 Traffic	Fixed signal timing	Wait and queue length
SC05 Timetable	First-fit timetable	Hard and soft violations
SC06 Route	Nearest-neighbour route	Distance and violations
SC07 Irrigation	Fixed moisture threshold	Water and plant-stress proxy
SC08 Maintenance	Fixed alarm threshold	Failure recall and false alarms
SC09 Credit	Simple score/majority rule	Recall and false approvals
SC10 Energy	Fixed/first-come schedule	Cost, peak and violations

Every team must use the **same inputs or scenarios** and **same measure** for baseline and Step 3 method. Report failure cases and limitations. Do not claim improvement unless the comparison shows it.

Final Step 4 self-check

- The branch starts from Step 3 and contains all earlier files.
- The baseline rule was chosen from training data only, with tie handling documented.
- Saved ANN predictions match validation/test rows exactly; no test tuning was done.
- Both methods have accuracy, precision, recall and F1 on the same rows.
- Paired predictions, comparison CSV and graph are committed.
- The notebook ran fully and its outputs are saved.
- STEP-4.md contains actual numbers, interpretation, limitations and member contributions.
- GitHub shows the Step 4 branch and meaningful commit.
- For an actual allocated team, the portal shows a Step 4 submission by the deadline if possible; do not submit SC11 as a student team.

One sentence to remember

Step 4 is not about making the baseline look bad. It is about making a fair, repeatable comparison and explaining the evidence honestly.

Reference material: SC11 Step 3 branch at https://github.com/vipulshahi-ai/SC11-Water-Quality-Potability-Example/tree/step-3-ann-model and the official Step 4 student portal guide at https://galgotias-projects.github.io/Soft-Computing-Project-Selection/step-4-guide.html.

Appendix A - reusable STEP-4.md template

Reuse these headings and checks. Replace every bracketed prompt with your own project evidence; do not submit a blank template.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
# Step 4 — Fair comparison report

> Template for SC01–SC10. Replace every bracketed prompt. Do not submit this unfinished template or copy SC

## Team and project

- Project: [code and title]
- Team: [team name and members]
- Repository branch and commit: [branch name and commit link]

## Question

What does your Step 3 method do, and what simple baseline will you compare it with? State one specific ques

## Data and scenarios

- Source and citation: [link]
- Training data or development scenarios: [count and purpose]
- Validation and test data or scenarios: [counts and how separated]
- Target/output and relevant input fields: [names and meaning]
- Generated scenarios, if any: [label clearly and describe generation]

## Baseline rule

Describe a simple, reproducible rule that fits your own project. State every threshold, tie rule and constr

## Fair-comparison checks

- [ ] Both methods answer the same question and use the same units.
- [ ] Both methods are tested on identical held-out rows or scenarios.
- [ ] Row identity, original labels and ordering were verified.
- [ ] The baseline did not use test answers to choose its rule.
- [ ] Appropriate metrics were chosen before comparing scores.

## Results
```

Appendix A - reusable STEP-4.md template (continued)

```
| Split | Measure and unit | Baseline | Step 3 method | Difference |
| --- | --- | ---: | ---: | ---: |
| Validation | [metric 1] | [actual] | [actual] | [actual] |
| Test | [metric 1] | [actual] | [actual] | [actual] |
| Test | [metric 2] | [actual] | [actual] | [actual] |

Do not invent numbers. For cost, energy, violations or waiting time, lower may be better; explain the direc

## Interpretation

- What improved, what worsened, and by how much? [Use actual output.]
- Is the difference meaningful for the project goal? [Explain.]
- Which examples failed or disagree? [Point to evidence.]

## Limitations and responsible use

State dataset limitations, potential bias, failure cases and real-world safety or fairness boundaries relev

## Reproducibility and evidence

- Script: [path]
- Executed notebook: [path]
- Metrics/comparison table: [path]
- Paired predictions or scenario outcomes: [path]
- Chart or visualization: [path]
- Instructions to rerun using VS Code: [brief steps]

## Team contributions

- [Member name]: [specific work and evidence]
- [Member name]: [specific work and evidence]
- [Member name]: [specific work and evidence]
- [Fourth member, if applicable]: [specific work and evidence]
```

Appendix B - SC11 report example

Read how SC11 documents its question and method. Your team must report its own data and results.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
# SC11 Step 4 - Fair baseline comparison

## Question
On the same held-out validation and test rows, does the Step 3 ANN predict
potability better than a simple training-majority baseline?

## Baseline rule
Count `Potability` labels in `data/processed/train.csv` only. Always predict
the more common label. If tied, choose `0`. Never look at validation/test labels
while choosing this rule.

## Fairness checks
- Step 3 ANN predictions come from `step3_validation_predictions.csv` and
  `step3_test_predictions.csv`.
- `src/compare_baseline.py` checks row count, feature columns and true labels
  against the same Step 2 validation/test CSVs before scoring.
- The held-out test set is not used to tune either method.
- Generated Step 2 scenarios are kept separate; they are not real labelled
  observations and are not included in this comparison.

## Results

These are the saved results from `data/reports/step4_baseline_metrics.csv`. Precision,
recall and F1 treat `Potability = 1` as the positive class.

| Split | Method | Rows | Accuracy | Precision | Recall | F1 |
|---|---|---:|---:|---:|---:|---:|
| Validation | Majority baseline | 384 | 0.5000 | 0.0000 | 0.0000 | 0.0000 |
| Validation | Step 3 ANN | 384 | 0.5443 | 0.5415 | 0.5781 | 0.5592 |
| Test | Majority baseline | 384 | 0.5000 | 0.0000 | 0.0000 | 0.0000 |
| Test | Step 3 ANN | 384 | 0.6276 | 0.6310 | 0.6146 | 0.6227 |

## Interpretation
- The training split has 894 records of each class. Because it is tied, the
  documented baseline predicts `0` for every record. It never predicts a
  potable case, so its precision, recall and F1 for class `1` are zero.
```

Appendix B - SC11 report example (continued)

```
- On validation, ANN accuracy is 0.5443 versus 0.5000 for the baseline: an
  increase of **0.0443 score points**. Validation F1 is 0.5592 versus 0.0000.
- On test, ANN accuracy is 0.6276 versus 0.5000: an increase of **0.1276 score
  points**. Test F1 is 0.6227 versus 0.0000. See
  `data/reports/step4_comparison.csv` for all four metric differences.
- The ANN is higher on every metric reported here; no measured metric became
  worse in this comparison. This does not establish that it is reliable for
  drinking-water decisions.
- The balanced class distribution makes the always-`0` baseline's accuracy
  0.5000, yet that same rule misses every class-`1` case. Accuracy alone hides
  this failure; precision, recall and F1 must be read alongside it.
- The cited labelled dataset has only 2,556 records, and some source values
  were missing and filled during Step 2. The 256 generated scenarios are not
  independent laboratory labels and are excluded from these scores. This
  comparison is limited to the available held-out data and one simple baseline.

## Evidence
- `src/compare_baseline.py`
- `notebooks/03_baseline_comparison.ipynb` executed with visible outputs
- `data/reports/step4_baseline_metrics.csv`
- `data/reports/step4_comparison.csv`
- `data/reports/step4_validation_paired_predictions.csv`
- `data/reports/step4_test_paired_predictions.csv`
- `results/step4_baseline_vs_ann.png`

## Safety and scope
This is an educational model comparison. Neither method can establish that
water is safe to drink. Laboratory analysis and the relevant official
standards are still required for any real-world decision. Step 4 is a fair
comparison, not the final Product V1 interface.

## Team contributions
SC11 is an instructor teaching example, not an allocated student team. There
are no student member contributions to claim here. Each allocated team must
replace this section with its real members' names and specific work before
submitting its own Step 4 report.
```

Appendix C - complete SC11 compare_baseline.py

Complete executable SC11 source for study. Adapt columns, rules, paths and metrics; do not submit it unchanged for SC01-SC10.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
"""SC11 Step 4: compare a training-only majority baseline with Step 3 ANN.

Run this file with VS Code's "Run Python File" button from the SC11 project.
Copy it to src/compare_baseline.py on a new Step 4 branch. The comparison
uses the saved Step 3 predictions; it does not retrain or tune the ANN.
"""

from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score


PROJECT_ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = PROJECT_ROOT / "data" / "processed"
REPORT_DIR = PROJECT_ROOT / "data" / "reports"
RESULTS_DIR = PROJECT_ROOT / "results"

FEATURES = [
    "ph", "Hardness", "Solids", "Chloramines", "Sulfate",
    "Conductivity", "Organic_carbon", "Trihalomethanes", "Turbidity",
]
TARGET = "Potability"
METRICS = ("accuracy", "precision", "recall", "f1_score")


def read_checked_csv(path, required_columns):
    if not path.is_file():
        raise FileNotFoundError(f"Missing {path}. Finish Step 2 and Step 3 first.")
    frame = pd.read_csv(path)
    missing = [name for name in required_columns if name not in frame.columns]
    if missing:
        raise ValueError(f"{path.name} is missing columns: {missing}")
    if frame[required_columns].isna().any().any():
        raise ValueError(f"{path.name} contains blank required values.")
    return frame

def check_binary_labels(values, description):
    labels = set(pd.Series(values).unique())
    if not labels.issubset({0, 1}):
        raise ValueError(f"{description} must contain only labels 0 and 1: {labels}")


def score(actual, predicted, split, method):
    return {
        "split": split,
        "method": method,
        "rows": len(actual),
        "accuracy": round(accuracy_score(actual, predicted), 4),
```

```
        "precision": round(precision_score(actual, predicted, zero_division=0), 4),
        "recall": round(recall_score(actual, predicted, zero_division=0), 4),
        "f1_score": round(f1_score(actual, predicted, zero_division=0), 4),
    }


def make_chart(metric_rows):
    figure, axes = plt.subplots(1, 2, figsize=(11, 4), sharey=True)
    for axis, split in zip(axes, ("validation", "test")):
        section = metric_rows.set_index(["split", "method"])
        baseline = section.loc[(split, "Majority baseline"), list(METRICS)].astype(float)
        ann = section.loc[(split, "Step 3 ANN"), list(METRICS)].astype(float)
        positions = range(len(METRICS))
        axis.bar(
            [x - 0.18 for x in positions], baseline,
            width=0.36, label="Baseline", color="#777777",
        )
        axis.bar(
            [x + 0.18 for x in positions], ann,
            width=0.36, label="ANN", color="#222222",
        )
        axis.set_xticks(list(positions), ["Accuracy", "Precision", "Recall", "F1"])
        axis.set_ylim(0, 1)
        axis.set_title(split.title())
        axis.grid(axis="y", alpha=0.25)
    axes[0].set_ylabel("Score (higher is better)")
    axes[1].legend(loc="upper right")
    figure.suptitle("SC11 Step 4: Same data, two methods")
    figure.tight_layout()
    chart_path = RESULTS_DIR / "step4_baseline_vs_ann.png"
    figure.savefig(chart_path, dpi=180)
    plt.close(figure)
    return chart_path


def main():
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)

    train = read_checked_csv(DATA_DIR / "train.csv", FEATURES + [TARGET])
    check_binary_labels(train[TARGET], "Training target")
    counts = train[TARGET].value_counts()
    # A tie is resolved as 0. The choice is made from training labels only.
    majority_label = 0 if counts.get(0, 0) >= counts.get(1, 0) else 1
    print("SC11 STEP 4 - FAIR BASELINE COMPARISON")
    print(f"Training rows: {len(train)}; class counts: {counts.to_dict()}")
    print(f"Majority baseline always predicts: {majority_label}")

    metric_rows = []
    comparison_rows = []
    for split in ("validation", "test"):
        actual = read_checked_csv(DATA_DIR / f"{split}.csv", FEATURES + [TARGET])
        saved = read_checked_csv(
            REPORT_DIR / f"step3_{split}_predictions.csv",
            FEATURES + [TARGET, "ann_prediction"],
        )
        if len(saved) != len(actual):
            raise ValueError(f"{split}: Step 3 predictions have a different row count.")
        try:
            pd.testing.assert_frame_equal(
                actual[FEATURES + [TARGET]].reset_index(drop=True),
```

```
                saved[FEATURES + [TARGET]].reset_index(drop=True),
                check_dtype=False, check_exact=False, rtol=1e-8, atol=1e-10,
            )
        except AssertionError as error:
            raise ValueError(
                f"{split}: Step 3 predictions do not match the Step 2 split. "
                "Do not compare different rows or reorder the CSV files."
            ) from error
        check_binary_labels(actual[TARGET], f"{split} target")
        check_binary_labels(saved["ann_prediction"], f"{split} ANN predictions")

        baseline_predictions = pd.Series(majority_label, index=actual.index)
        baseline_result = score(
            actual[TARGET], baseline_predictions, split, "Majority baseline"
        )
        ann_result = score(actual[TARGET], saved["ann_prediction"], split, "Step 3 ANN")
        metric_rows.extend((baseline_result, ann_result))

        for metric in METRICS:
            comparison_rows.append({
                "split": split,
                "metric": metric,
                "baseline": baseline_result[metric],
                "ann": ann_result[metric],
                "ann_minus_baseline": round(
                    ann_result[metric] - baseline_result[metric], 4
                ),
            })

        paired = actual[FEATURES + [TARGET]].copy()
        paired["baseline_prediction"] = majority_label
        paired["ann_prediction"] = saved["ann_prediction"].to_numpy()
        paired.to_csv(REPORT_DIR / f"step4_{split}_paired_predictions.csv", index=False)

    metrics = pd.DataFrame(metric_rows)
    comparison = pd.DataFrame(comparison_rows)
    metrics.to_csv(REPORT_DIR / "step4_baseline_metrics.csv", index=False)
    comparison.to_csv(REPORT_DIR / "step4_comparison.csv", index=False)
    chart_path = make_chart(metrics)

    print("\nBoth methods use exactly the same validation/test rows.")
    print(metrics.to_string(index=False))
    print("\nANN minus baseline (positive means ANN is higher):")
    print(comparison.to_string(index=False))
    print(f"\nSaved chart: {chart_path}")
    print(f"Saved CSV evidence: {REPORT_DIR}")
    print("STEP 4 COMPLETED")
    return metrics, comparison


if __name__ == "__main__":
    main()
```

Appendix D - SC11 notebook cell 1 (markdown)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
# SC11 Step 4 - fair baseline comparison
This notebook is a worked teaching example. The baseline always predicts
the most common training label; the ANN predictions are the saved Step 3
outputs. Both are compared on exactly the same validation and test rows.
This is **not** a water-safety test.
```

Appendix D - SC11 notebook cell 2 (code)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
from pathlib import Path
import sys

# VS Code may start a notebook in the repository root or in notebooks/.
ROOT = next(
    (p for p in [Path.cwd(), *Path.cwd().parents]
     if (p / 'src' / 'compare_baseline.py').is_file()),
    None,
)
if ROOT is None:
    raise FileNotFoundError(
        'Open the SC11 repository folder in VS Code and create '
        'src/compare_baseline.py first.'
    )
sys.path.insert(0, str(ROOT))
print('Project folder:', ROOT)
print('Step 3 metrics exist:',
      (ROOT / 'data' / 'reports' / 'step3_ann_metrics.csv').is_file())
```

Appendix D - SC11 notebook cell 3 (markdown)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
## Run the same reproducible comparison
The Python file checks that saved ANN predictions match the Step 2 split
row-for-row. It stops with an error if files are missing or mismatched.
```

Appendix D - SC11 notebook cell 4 (code)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
from src import compare_baseline
metrics, comparison = compare_baseline.main()
```

Appendix D - SC11 notebook cell 5 (markdown)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
## Read the comparison
`ann_minus_baseline` is a score difference, not a percentage improvement.
A positive number means the ANN scored higher for that metric on the same split.
```

Appendix D - SC11 notebook cell 6 (code)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
from IPython.display import display, Image
display(metrics)
display(comparison)
display(Image(filename=str(ROOT / 'results' / 'step4_baseline_vs_ann.png')))
```

Appendix D - SC11 notebook cell 7 (markdown)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
## Inspect a few paired predictions
The baseline and ANN columns must refer to the same original row and known
label. Do not treat generated testing scenarios as real labelled samples.
```

Appendix D - SC11 notebook cell 8 (code)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
import pandas as pd
paired = pd.read_csv(ROOT / 'data' / 'reports' / 'step4_test_paired_predictions.csv')
display(
    paired[['Potability', 'baseline_prediction', 'ann_prediction']].head(10)
)
print('Test rows compared:', len(paired))
```

Appendix D - SC11 notebook cell 9 (markdown)

Study this cell's purpose, then create corresponding analysis for your own project. Run All and save visible outputs before committing.

Complete source is shown for study. Reuse the workflow, not SC11-specific data, rules, model or results, in another project.

```
## Explain the result in your own words
Record the baseline rule, training-only label count, validation/test sizes,
all four metrics, limits of the dataset, and the fact that a classifier
cannot certify drinking water. Adapt this workflow - not the SC11 code -
for your own allotted project.
```